

Examples:

1. A matrix product.

> A := <<1|-2|3>>, <0|1|1>>; B := Matrix(3, 2, symbol=b); C := A . B;

$$A := \begin{bmatrix} 1 & -2 & 3 \\ 0 & 1 & 1 \end{bmatrix}, \quad B := \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix},$$

$$C := \begin{bmatrix} b_{11} - 2b_{21} + 3b_{31} & b_{12} - 2b_{22} + 3b_{32} \\ b_{21} + b_{31} & b_{22} + b_{32} \end{bmatrix}.$$

2. A Gram–Schmidt calculation revisited.

If u_1, u_2 are $m \times 1$ column matrices, then the Gram–Schmidt process is often written in textbooks as

$$w_2 = u_2 - \frac{u_2^T u_1}{u_1^T u_1} u_1.$$

Notice, however, that $u_2^T u_1$ and $u_1^T u_1$ are strictly 1×1 matrices. Textbooks often skip over the conversion of $u_2^T u_1$ from a 1×1 matrix to a scalar. Maple, in contrast, does not convert automatically.

Transcribing the printed formula into Maple will cause an error. Here is the way to do it, reusing the earlier numerical data.

> u1 := <<3, 0, 4>>; u2 := <<2, 1, 1>>; r := u2^%T . u1;
s := u1^%T . u1;

$$u1 := \begin{bmatrix} 3 \\ 0 \\ 4 \end{bmatrix}, \quad u2 := \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}, \quad r := [10], \quad s := [25].$$

Notice the brackets in the values of r and s because they are matrices. Since $r[1, 1]$ and $s[1, 1]$ are scalars, we write

> w2 := u2 - r[1, 1]/s[1, 1]*u1;

and reobtain the result from Example 3 in Section 72.2. Alternatively, $u1$ and $u2$ can be converted to **Vectors** first and then used to form a proper scalar inner product.

> r := u2[1..-1, 1] . u1[1..-1, 1]; s := u1[1..-1, 1] . u1[1..-1, 1];
w2 := u2 - r/s*u1;

$$r := 10, \quad s := 25, \quad w2 := \begin{bmatrix} 4/5 \\ 1 \\ -3/5 \end{bmatrix}.$$

3. Vector–Matrix and Matrix–Vector products.

Many textbooks equate a column vector and a one-column matrix, but this is not generally so in Maple. Thus

> b := <1, 2>; B := <<1, 2>>; C := <<4|5|6>>;

$$b := \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad B := \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad C := \begin{bmatrix} 4 & 5 & 6 \end{bmatrix}.$$

Only the product $B . C$ is defined, and the product $b . C$ causes an error.

> B . C

$$\begin{bmatrix} 4 & 5 & 6 \\ 8 & 10 & 12 \end{bmatrix}.$$

The rules for mixed products are

```
Vector[row]( n ) . Matrix( n, m )          = Vector[row]( m )
Matrix( n, m ) . Vector[column]( m )       = Vector[column]( n )
```

The combinations `Vector(n).Matrix(1, m)` and `Matrix(m, 1).Vector(row)(n)` cause errors. If users do not want this level of rigor, then the easiest thing to do is to use only the `Matrix` declaration.

4. Working with matrices containing complex elements.

First, notation: In linear algebra, I is commonly used for the identity matrix. This corresponds to the `eye` function in MATLAB. However, by default, Maple uses I for the imaginary unit, as seen in section 72.2. We can, however, use I for an identity matrix by changing the imaginary unit to something else, say `_i`.

```
> interface( imaginaryunit=_i ):
```

As the saying goes: An `_i` for an I and an I for an eye.

Now we can calculate eigenvalues using notation similar to introductory textbooks.

```
> A := <<1,2>>|<-2,1>>; I := IdentityMatrix( 2 );
p := Determinant ( x*I-A );
```

$$A := \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix}, \quad I := \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad p := x^2 - 2x + 5.$$

Solving $p = 0$, we obtain eigenvalues $1 + 2i, 1 - 2i$. With the above setting of `imaginaryunit`, Maple will print these values as $1+2_i, 1-2_i$, but we have translated back to standard mathematical i , where $i^2 = -1$.

5. Moore–Penrose inverse. Consider `M := Matrix(3,2,[[1,1],[a,a^2],[a^2,a]]);`, a 3×2 matrix containing a symbolic parameter a . We compute its Moore–Penrose pseudoinverse and a proviso guaranteeing correctness by the command `> (Mi, p) := MatrixInverse\ (M, method=pseudo, output=[inverse, proviso]);` which assigns the 2×3 pseudoinverse to `Mi` and an expression, which if nonzero guarantees that `Mi` is the correct (unique) Moore–Penrose pseudoinverse of M . Here we have

$$Mi := \begin{bmatrix} (2 + 2a^3 + a^2 + a^4)^{-1} & -\frac{a^3 + a^2 + 1}{a(a^5 + a^4 - a^3 - a^2 + 2a - 2)} & \frac{a^4 + a^3 + 1}{a(a^5 + a^4 - a^3 - a^2 + 2a - 2)} \\ (2 + 2a^3 + a^2 + a^4)^{-1} & \frac{a^4 + a^3 + 1}{a(a^5 + a^4 - a^3 - a^2 + 2a - 2)} & -\frac{a^3 + a^2 + 1}{a(a^5 + a^4 - a^3 - a^2 + 2a - 2)} \end{bmatrix}$$

and $p = a^2 - a$. Thus, if $a \neq 0$ and $a \neq 1$, the computed pseudoinverse is correct. By separate computations we find that the pseudoinverse of $M|_{a=0}$ is

$$\begin{bmatrix} 1/2 & 0 & 0 \\ 1/2 & 0 & 0 \end{bmatrix}$$

and that the pseudoinverse of $M|_{a=1}$ is

$$\begin{bmatrix} 1/6 & 1/6 & 1/6 \\ 1/6 & 1/6 & 1/6 \end{bmatrix}$$

and moreover that these are not special cases of the generic answer returned previously. In a certain sense this is obvious: the Moore–Penrose inverse is *discontinuous*, even for square matrices (consider $(A - \lambda I)^{-1}$, for example, as $\lambda \rightarrow$ an eigenvalue of A).

72.4 Arrays

Before describing Maple's `Array` structure, it is useful to say why Maple distinguishes between an `Array` and a `Vector` or `Matrix`, when other books and software systems do not. In linear algebra, two different types of operations are performed with vectors or matrices. The first type is described in Sections 72.2 and 72.3, and comprises operations derived from the mathematical structure of vector spaces. The other type comprises operations that treat vectors or matrices as data arrays; they manipulate the individual elements directly. As an example, consider dividing the elements of `Array [1, 3, 5]` by the elements of `[7, 11, 13]` to obtain `[1/7, 3/11, 5/13]`.

The distinction between the operations can be made in two places: in the name of the operation or the name of the object. In other words we can overload the data objects or overload the operators. Systems such as MATLAB choose to leave the data object unchanged, and define separate operators. Thus, in MATLAB the statements `[1, 3, 5]/[7, 11, 13]` and `[1, 3, 5]./[7, 11, 13]` are different because of the operators. In contrast, Maple chooses to make the distinction in the data object, as will now be described.

Facts:

1. The Maple `Array` is a general data structure akin to arrays in other programming languages.
2. An array can have up to 63 indices and each index can lie in any integer range.
3. The description here only addresses the overlap between Maple `Array` and `Vector`.

Caution: A Maple `Array` might look the same as a vector or matrix when printed.

Commands:

1. Generation of arrays.
 - `Array([x1, x2, ...])` Construct an array by listing its elements.
 - `Array( m..n )` Declare an empty 1-dimensional array indexed from m to n .
 - `Array( v )` Use an existing `Vector` to generate an array.
 - `convert( v, Array )` Convert a `Vector` v into an `Array`. Similarly, a `Matrix` can be converted to an `Array`. See the help file for `rtable_options` for advanced methods to convert efficiently, in-place.
2. Operations (memory/stack limitations may restrict operations).
 - `a ^ n` Raise each element of a to power n .
 - `a * b`, `a + b`, `a - b` Multiply (add, subtract) elements of b by (to, from) elements of a .
 - `a / b` Divide elements of a by elements of b . Division by zero will produce undefined or infinity (or exceptions can be caught by user-set traps; see the help file for `Numeric_Events`).

Examples:

1. Array arithmetic.


```
> simplify( (Array([25, 9, 4]) * Array(1..3, x->x^2-1) ) / Array(<5, 3, 2>\
) )^(1/2) );
```

[0, 3, 4]

2. Getting Vectors and Arrays to do the same thing.

```
> Transpose( map(x->x*x,<1,2,3> ) ) - convert(Array( [1,2,3] )^2,\
Vector);
```

[0,0,0]

72.5 Equation Solving and Matrix Factoring

Cautions:

1. If a matrix contains exact numerical entries, typically integers or rationals, then the material studied in introductory textbooks transfers to a computer algebra system without special considerations. However, if a matrix contains symbolic entries, then the fact that computations are completed without the user seeing the intermediate steps can lead to unexpected results.
2. Some of the most popular matrix functions are discontinuous when applied to matrices containing symbolic entries. Examples are given below.
3. Some algorithms taught to educate students about the concepts of linear algebra often turn out to be ill-advised in practice: computing the characteristic polynomial and then solving it to find eigenvalues, for example; using Gaussian elimination without pivoting on a matrix containing floating-point entries, for another.

Commands:

1. `LinearSolve( A, B )` The vector or matrix X satisfying $AX = B$.
2. `BackwardSubstitute( A, B )`, `ForwardSubstitute( A, B )` The vector or matrix X satisfying $AX = B$ when A is upper or lower triangular (echelon) form, respectively.
3. `ReducedRowEchelonForm( A )`. The reduced row-echelon form (RREF) of the matrix A . For matrices with symbolic entries, see the examples below for recommended usage.
4. `Rank( A )` The rank of the matrix A . *Caution:* If A has floating-point entries, see the section below on Numerical Linear Algebra. On the other hand, if A contains symbolic entries, then the rank may change discontinuously and the generic answer returned by `Rank` may be incorrect for some specializations of the parameters.
5. `NullSpace( A )` The nullspace (kernel) of the matrix A . *Caution:* If A has floating-point entries, see the section below on Numerical Linear Algebra. Again on the other hand, if A contains symbolic entries, the nullspace may change discontinuously and the generic answer returned by `NullSpace` may be incorrect for some specializations of the parameters.
6. `( P, L, U, R ) := LUDecomposition( A, method='RREF' )` The $PLUR$, or Turing, factors of the matrix A . See examples for usage.
7. `( P, L, U ) := LUDecomposition( A )` The PLU factors of a matrix A , when the RREF R is not needed. This is usually the case for a Turing factoring where R is guaranteed (or known *a priori*) to be I , the identity matrix, for all values of the parameters.
8. `( Q, R ) := QRDecomposition( A, fullspan )` The QR factors of the matrix A . The option `fullspan` ensures that Q is square.
9. `SingularValues( A )` See Section 72.8, Numerical Linear Algebra.
10. `ConditionNumber( A )` See Section 72.8, Numerical Linear Algebra.

Examples:

1. Need for Turing factoring.

One of the strengths of Maple is computation with symbolic quantities. When standard linear algebra methods are applied to matrices containing symbolic entries, the user must be aware of new mathematical features that can arise. The main feature is the discontinuity of standard matrix

functions, such as the reduced row-echelon form and the rank, both of which can be discontinuous. For example, the matrix

$$B = A - \lambda I = \begin{bmatrix} 7 - \lambda & 4 \\ 6 & 2 - \lambda \end{bmatrix}$$

has the reduced row-echelon form

$$\text{ReducedRowEchelonForm}(B) = \begin{cases} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & \lambda \neq -1, 10 \\ \begin{bmatrix} 1 & -4/3 \\ 0 & 0 \end{bmatrix} & \lambda = 10, \\ \begin{bmatrix} 1 & 1/2 \\ 0 & 0 \end{bmatrix} & \lambda = -1. \end{cases}$$

Notice that the function is discontinuous precisely at the interesting values of λ . Computer algebra systems in general, and Maple in particular, return “generic” results. Thus, in Maple, we have

```
> B := << 7-x | 4 >, < 6 | 2-x >>;
      B :=  $\begin{bmatrix} 7-x & 4 \\ 6 & 2-x \end{bmatrix},$ 
> ReducedRowEchelonForm( B )
```

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

This difficulty is discussed at length in [CJ92] and [CJ97]. The recommended solution is to use Turing factoring (generalized PLU decomposition) to obtain the reduced row-echelon form with provisos. Thus, for example,

```
> A := <<1 | -2 | 3 | sin(x) >, <1 | 4*cos(x) | 3 | 3*sin(x) >, <-1 | 3 | cos(x) - 3 | \
cos(x) >>;
```

$$A := \begin{bmatrix} 1 & -2 & 3 & \sin x \\ 1 & 4 \cos x & 3 & 3 \sin x \\ -1 & 3 & \cos x - 3 & \cos x \end{bmatrix}.$$

```
> ( P, L, U, R ) := LUDecomposition( A, method='RREF' );
The generic reduced row-echelon form is then given by
```

$$R = \begin{bmatrix} 1 & 0 & 0 & (2 \sin x \cos x - 3 \sin x - 6 \cos x - 3)/(2 \cos x + 1) \\ 0 & 1 & 0 & \sin x/(2 \cos x + 1) \\ 0 & 0 & 1 & (2 \cos x + 1 + 2 \sin x)/(2 \cos x + 1) \end{bmatrix}$$

This shows a visible failure when $2 \cos x + 1 = 0$, but the other discontinuity is invisible, and requires the U factor from the Turing ($PLUR$) factors,

$$U = \begin{bmatrix} 1 & -2 & 3 \\ 0 & 4 \cos x + 2 & 0 \\ 0 & 0 & \cos x \end{bmatrix}$$

to see that the case $\cos x = 0$ also causes failure. In both cases (meaning the cases $2 \cos x + 1 = 0$ and $\cos x = 0$), the RREF must be recomputed to obtain the singular cases correctly.

2. QR factoring.

Maple does not offer column pivoting, so in pathological cases the factoring may not be unique, and will vary between software systems. For example,

```
> A := <<0,0>|<5,12>>: QRDecomposition( A, fullspan )
```

$$\begin{bmatrix} 5/13 & 12/13 \\ 12/13 & -5/13 \end{bmatrix}, \begin{bmatrix} 0 & 13 \\ 0 & 0 \end{bmatrix}.$$

72.6 Eigenvalues and Eigenvectors

Facts:

1. In exact arithmetic, explicit expressions are not possible in general for the eigenvalues of a matrix of dimension 5 or higher.
2. When it has to, Maple represents polynomial roots (and, hence, eigenvalues) *implicitly* by the `RootOf` construct. Expressions containing `RootOf`s can be simplified and evaluated numerically.

Commands:

1. `Eigenvalues( A )` The eigenvalues of matrix A .
2. `Eigenvectors( A )` The eigenvalues and corresponding eigenvectors of A .
3. `CharacteristicPolynomial( A, 'x' )` The characteristic polynomial of A expressed using the variable x .
4. `JordanForm( A )` The Jordan form of the matrix A .

Examples:

1. Simple eigensystem computation.

```
> Eigenvectors( <<7,6>|<4,2>> );
```

$$\begin{bmatrix} -1 \\ 10 \end{bmatrix}, \begin{bmatrix} -1/2 & 4/3 \\ 1 & 1 \end{bmatrix}.$$

So the eigenvalues are -1 and 10 with the corresponding eigenvectors $[-1/2, 1]^T$ and $[4/3, 1]^T$.

2. A defective matrix.

If the matrix is defective, then by convention the matrix of “eigenvectors” returned by Maple contains one or more columns of zeros.

```
> Eigenvectors( <<1,0>|<1,1>> );
```

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

3. Larger systems.

For larger matrices, the eigenvectors will use the Maple `RootOf` construction,

$$A := \begin{bmatrix} 3 & 1 & 7 & 1 \\ 5 & 6 & -3 & 5 \\ 3 & -1 & -1 & 0 \\ -1 & 5 & 1 & 5 \end{bmatrix}.$$

```
> ( L, V ) := Eigenvectors( A );
```

The colon suppresses printing. The vector of eigenvalues is returned as

> L;

$$\begin{bmatrix} \text{RootOf}(\mathcal{Z}^4 - 13\mathcal{Z}^3 - 4\mathcal{Z}^2 + 319\mathcal{Z} - 386, \text{index} = 1) \\ \text{RootOf}(\mathcal{Z}^4 - 13\mathcal{Z}^3 - 4\mathcal{Z}^2 + 319\mathcal{Z} - 386, \text{index} = 2) \\ \text{RootOf}(\mathcal{Z}^4 - 13\mathcal{Z}^3 - 4\mathcal{Z}^2 + 319\mathcal{Z} - 386, \text{index} = 3) \\ \text{RootOf}(\mathcal{Z}^4 - 13\mathcal{Z}^3 - 4\mathcal{Z}^2 + 319\mathcal{Z} - 386, \text{index} = 4) \end{bmatrix}.$$

This, of course, simply reflects the characteristic polynomial:

> CharacteristicPolynomial(A, 'x');

$$x^4 - 13x^3 - 4x^2 + 319x - 386$$

The Eigenvalues command solves a 4th degree characteristic polynomial explicitly in terms of radicals unless the option `implicit` is used.

4. Jordan form. *Caution:* As with the reduced row-echelon form, the Jordan form of a matrix containing symbolic elements can be discontinuous. For example, given

$$A = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix},$$

> (J, Q) := JordanForm(A, output=['J', 'Q']);

$$J, Q := \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} t & 0 \\ 0 & 1 \end{bmatrix}$$

with $A = QJQ^{-1}$. Note that Q is invertible precisely when $t \neq 0$. This gives a proviso on the correctness of the result: J will be the Jordan form of A only for $t \neq 0$, which we see is the generic case returned by Maple.

Caution: Exact computation has its limitations, even without symbolic entries. If we ask for the Jordan form of the matrix

$$B = \begin{bmatrix} -1 & 4 & -1 & -14 & 20 & -8 \\ 4 & -5 & -63 & 203 & -217 & 78 \\ -1 & -63 & 403 & -893 & 834 & -280 \\ -14 & 203 & -893 & 1703 & -1469 & 470 \\ 20 & -217 & 834 & -1469 & 1204 & -372 \\ -8 & 78 & -280 & 470 & -372 & 112 \end{bmatrix},$$

a relatively modest 6×6 matrix with a triple eigenvalue 0, then the transformation matrix Q as produced by Maple has entries over 35,000 characters long. Some scheme of compression or large expression management is thereby mandated.

72.7 Linear Algebra with Modular Arithmetic in Maple

There is a subpackage, `LinearAlgebra[Modular]`, designed for programmatic use, that offers access to modular arithmetic with matrices and vectors.

Facts:

1. The subpackage can be loaded by issuing the command
 > with(LinearAlgebra[Modular]); which gives access to the commands
`[AddMultiple, Adjoint, BackwardSubstitute, Basis, CharacteristicPolynomial, ChineseRemainder, Copy, Create, Determinant, Fill,`

ForwardSubstitute, Identity, Inverse, LUApply, LUDecomposition, LinIntSolve, MatBasis, MatGcd, Mod, Multiply, Permute, Random, Rank, RankProfile, RowEchelonTransform, RowReduce, Swap, Transpose, ZigZag]

2. Arithmetic can be done modulo a prime p or, in some cases, a composite modulus m .
3. The relevant matrix and vector datatypes are `integer[4]`, `integer[8]`, `integer[]`, and `float[8]`. Use of the correct datatype can improve efficiency.

Examples:

```
> p := 13;
> A := Mod( p, Matrix([[1,2,3],[4,5,6],[7,8,-9]]), integer[4] );
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 4 \end{bmatrix},$$

```
> Mod( p, MatrixInverse( A ), integer[4] );
```

$$\begin{bmatrix} 12 & 8 & 5 \\ 0 & 11 & 3 \\ 5 & 3 & 5 \end{bmatrix}$$

Cautions:

1. This is not to be confused with the `mod` utilities, which together with the inert `Inverse` command, can also be used to calculate inverses in a modular way.
2. One must always specify the datatype in `Modular` commands, or a cryptic error message will be generated.

72.8 Numerical Linear Algebra in Maple

The above sections have covered the use of Maple for exact computations of the types met during a standard first course on linear algebra. However, in addition to exact computation, Maple offers a variety of floating-point numerical linear algebra support.

Facts:

1. Maple can compute with either “hardware floats” or “software floats.”
2. A hardware float is IEEE double precision, with a mantissa of (approximately) 15 decimal digits.
3. A software float has a mantissa whose length is set by the Maple variable `Digits`.

Cautions:

1. If an integer is typed with a decimal point, then Maple treats it as a software float.
2. Software floats are significantly slower than hardware floats, even for the same precision.

Commands:

1. `Matrix( n, m, datatype=float[8] )` An $n \times m$ matrix of hardware floats (initialization data not shown). The elements must be real numbers. The 8 refers to the number of bytes used to store the floating point real number.
2. `Matrix( n, m, datatype=complex(float[8] ) )` An $n \times m$ matrix of hardware floats, including complex hardware floats. A complex hardware float takes two 8-byte storage locations.

3. `Matrix( n, m, datatype=sfloat )` An $n \times m$ matrix of software floats. The entries must be real and the precision is determined by the value of `Digits`.
4. `Matrix( n, m, datatype=complex(sfloat) )` As before with complex software floats.
5. `Matrix( n, m, shape=symmetric )` A matrix declared to be symmetric. Maple can take advantage of shape declarations such as this.

Examples:

1. A characteristic surprise.

When asked to compute the characteristic polynomial of a floating-point matrix, Maple first computes eigenvalues (by a good numerical method) and then presents the characteristic polynomial in factored form, with good approximate roots. Thus,

```
> CharacteristicPolynomial( Matrix( 2, 2, [[666,667],[665,666]], \
    datatype=float[8]), 'x' );
```

$$(x - 1331.99924924882612 - 0.0i)(x - 0.000750751173882235889 - 0.0i).$$

Notice the signed zero in the imaginary part; though the roots in this case are real, approximate computation of the eigenvalues of a nonsymmetric matrix takes place over the complex numbers. (n.b.: The output above has been edited for clarity.)

2. Symmetric matrices.

If Maple knows that a matrix is symmetric, then it uses appropriate routines. Without the symmetric declaration, the calculation is

```
> Eigenvalues( Matrix( 2, 2, [[1,3],[3,4]], datatype=float[8]) );
```

$$\begin{bmatrix} -0.854101966249684707 + 0.0i \\ 5.85410196624968470 + 0.0i \end{bmatrix}.$$

With the declaration, the computation is

```
> Eigenvalues( Matrix( 2, 2, [[1,3],[3,4]], shape=symmetric,
    datatype=float[8]) );
```

$$\begin{bmatrix} -0.854101966249684818 \\ 5.85410196624968470 \end{bmatrix}.$$

Cautions: Use of the `shape=symmetric` declaration will force Maple to treat the matrix as being symmetric, even if it is not.

3. Printing of hardware floats.

Maple prints hardware floating-point data as 18-digit numbers. This does not imply that all 18 digits are correct; Maple prints the hardware floats this way so that a cycle of converting from binary to decimal and back to binary will return to exactly the starting binary floating-point number. Notice in the previous example that the last 3 digits differ between the two function calls. In fact, neither set of digits is correct, as a calculation in software floats with higher precision shows:

```
> Digits := 30: Eigenvalues( Matrix( 2, 2, [[1,3],[3,4]], \
    datatype=sfloat, shape=symmetric ) );
```

$$\begin{bmatrix} -0.854101966249684544613760503093 \\ 5.85410196624968454461376050310 \end{bmatrix}.$$

4. NullSpace. Consider

```
> B := Matrix( 2, 2, [[666,667],[665,666]] ): A := Transpose(B).B;
```

We make a floating-point version of A by

```
> Af := Matrix( A, datatype=float[8] );
```

and then take the `NullSpace` of both A and Af . The nullspace of A is correctly returned as the empty set — A is not singular (in fact, its determinant is 1). The nullspace of Af is correctly returned as

$$\left\{ \begin{bmatrix} -0.707637442412755612 \\ 0.706575721416702662 \end{bmatrix} \right\}.$$

The answers are different — quite different — even though the matrices differ only in datatype. The surprising thing is that both answers are correct: Maple is doing the right thing in each case. See [Cor93] for a detailed explanation, but note that Af times the vector in the reported nullspace is about $3.17 \cdot 10^{-13}$ times the 2-norm of Af .

5. Approximate Jordan form (not!).¹

As noted previously, the Jordan form is discontinuous as a function of the entries of the matrix. This means that rounding errors may cause the computed Jordan form of a matrix with floating-point entries to be incorrect, and for this reason Maple refuses to compute the Jordan form of such a matrix.

6. Conditioning of eigenvalues.

To explore Maple's facilities for the conditioning of the unsymmetric matrix eigenproblem, consider the matrix "gallery(3)" from MATLAB.

```
> A := Matrix( [[-149,-50,-154], [537,180,546], [-27,-9,-25]] ):
```

The Maple command `EigenConditionNumbers` computes estimates for the reciprocal condition numbers of each eigenvalue, and estimates for the reciprocal condition numbers of the computed eigenvectors as well. At this time, there are no built-in facilities for the computation of the sensitivity of arbitrary invariant subspaces.

```
> E,V,rconds,rveconds := EigenConditionNumbers( A, output= \
['values', 'vectors', 'conditionvalues', 'conditionvectors'] ):
> seq( 1/rconds[i], i=1..3 );
417.6482708, 349.7497543, 117.2018824
```

A separate computation using the definition of the condition number of an eigentriplet $(\mathbf{y}^*, \lambda, \mathbf{x})$ (see Chapter 15) as

$$C_\lambda = \frac{\|\mathbf{y}^*\|_\infty \|\mathbf{x}\|_\infty}{|\mathbf{y}^* \cdot \mathbf{x}|}$$

gives exact condition numbers (in the infinity norm) for the eigenvalues 1, 2, 3 as 399, 252, and 147. We see that the estimates produced by `EigenConditionNumbers` are of the right order of magnitude.

72.9 Canonical Forms

There are several canonical forms in Maple: Jordan form, Smith form, Hermite form, and Frobenius form, to name a few. In this section, we talk only about the Smith form (defined in Chapter 6.5 and Chapter 23.2).

¹ An out-of-date, humorous reference.

Commands:

1. `SmithForm( B, output=['S', 'U', 'V'] )` Smith form of B .

Examples:

The Smith form of

$$B = \begin{bmatrix} 0 & -4y^2 & 4y & -1 \\ -4y^2 & 4y & -1 & 0 \\ 4y & -1 & 4(2y^2 - 2)y & 4(y^2 - 1)^2 y^2 - 2y^2 + 2 \\ -1 & 0 & 4(y^2 - 1)^2 y^2 - 2y^2 + 2 & -4(y^2 - 1)^2 y \end{bmatrix}$$

is

$$S = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/4 (2y^2 - 1)^2 & 0 \\ 0 & 0 & 0 & (1/64) (2y^2 - 1)^6 \end{bmatrix}.$$

Maple also returns two unimodular (over the domain $\mathbb{Q}[y]$) matrices $\mathbf{u}$ and $\mathbf{v}$ for which $A = U.S.V$.

72.10 Structured Matrices

Facts:

1. Computer algebra systems are particularly useful for computations with structured matrices.
2. User-defined structures may be programmed using *index functions*. See the help pages for details.
3. Examples of built-in structures include *symmetric*, *skew-symmetric*, *Hermitian*, *Vandermonde*, and *Circulant* matrices.

Examples:

Generalized Companion Matrices. Maple can deal with several kinds of generalized companion matrices. A generalized companion matrix² *pencil* of a polynomial $p(x)$ is a pair of matrices C_0, C_1 such that $\det(xC_1 - C_0) = 0$ precisely when $p(x) = 0$. Usually, in fact, $\det(xC_1 - C_0) = p(x)$, though in some definitions proportionality is all that is needed. In the case $C_1 = I$, the identity matrix, we have $C_0 = C(p(x))$ is the *companion matrix* of $p(x)$. MATLAB's `roots` function computes roots of polynomials by first computing the eigenvalues of the companion matrix, a venerable procedure only recently proved stable.

The generalizations allow direct use of alternative polynomial bases, such as the Chebyshev polynomials, Lagrange polynomials, Bernstein (Bézier) polynomials, and many more. Further, the generalizations allow the construction of generalized companion matrix pencils for *matrix polynomials*, allowing one to easily solve *nonlinear eigenvalue problems*.

We give three examples below.

If $p := 3 + 2x + x^2$, then `CompanionMatrix( p, x )` produces “the” (standard) companion matrix (also called Frobenius form companion matrix):

$$\begin{bmatrix} 0 & -3 \\ 1 & -2 \end{bmatrix}$$

²Sometimes known as “colleague” or “comrade” matrices, an unfortunate terminology that inhibits keyword search.

and it is easy to see that $\det(tI - C) = p(t)$. If instead

$$p := B_0^3(x) + 2B_1^3(x) + 3B_2^3(x) + 4B_3^3(x)$$

where $B_k^n(x) = \binom{n}{k}(1-x)^{n-k}(x+1)^k$ is the k th Bernstein (Bézier) polynomial of degree n on the interval $-1 \leq x \leq 1$, then `CompanionMatrix( p, x )` produces the pencil (note that this is not in Frobenius form)

$$C_0 = \begin{bmatrix} -3/2 & 0 & -4 \\ 1/2 & -1/2 & -8 \\ 0 & 1/2 & -\frac{52}{3} \end{bmatrix}$$

$$C_1 = \begin{bmatrix} 3/2 & 0 & -4 \\ 1/2 & 1/2 & -8 \\ 0 & 1/2 & -\frac{20}{3} \end{bmatrix}$$

(from a formula by Jonsson & Vavasis [JV05] and independently by J. Winkler [Win04]), and we have $p(x) = \det(xC_1 - C_0)$. Note that the program does not change the basis of the polynomial $p(x)$ of Equation (72.9) to the monomial basis (it turns out that $p(x) = 20 + 12x$ in the monomial basis, in this case: note that C_1 is singular). It is well-known that changing polynomial bases can be ill-conditioned, and this is why the routine avoids making the change.

Next, if we choose nodes $[-1, -1/3, 1/3, 1]$ and look at the degree 3 polynomial taking the values $[1, -1, 1, -1]$ on these four nodes, then `CompanionMatrix( values, nodes )` gives C_0 and C_1 where C_1 is the 5×5 identity matrix with the $(5, 5)$ entry replaced by 0, and

$$C_0 = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 \\ 0 & 1/3 & 0 & 0 & 1 \\ 0 & 0 & -1/3 & 0 & -1 \\ 0 & 0 & 0 & -1 & 1 \\ -\frac{9}{16} & \frac{27}{16} & -\frac{27}{16} & \frac{9}{16} & 0 \end{bmatrix}.$$

We have that $\det(tC_1 - C_0)$ is of degree 3 (in spite of these being 5×5 matrices), and that this polynomial takes on the desired values ± 1 at the nodes. Therefore, the finite eigenvalues of this pencil are the roots of the given polynomial. See [CWO4] and [Cor04] for example, for more information.

Finally, consider the nonlinear eigenvalue problem below: find the values of x such that the matrix C with $C_{ij} = T_0(x)/(i+j+1) + T_1(x)/(i+j+2) + T_2(x)/(i+j+3)$ is singular. Here $T_k(x)$ means the k th Chebyshev polynomial, $T_k(x) = \cos(k \cos^{-1}(x))$. We issue the command

```
> ( C0, C1 ) := CompanionMatrix( C, x );
```

from which we find

$$C0 = \begin{bmatrix} 0 & 0 & 0 & -2/15 & -1/12 & -\frac{2}{35} \\ 0 & 0 & 0 & -1/12 & -\frac{2}{35} & -1/24 \\ 0 & 0 & 0 & -\frac{2}{35} & -1/24 & -\frac{2}{63} \\ 1 & 0 & 0 & -1/4 & -1/5 & -1/6 \\ 0 & 1 & 0 & -1/5 & -1/6 & -1/7 \\ 0 & 0 & 1 & -1/6 & -1/7 & -1/8 \end{bmatrix}$$

and

$$C1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2/5 & 1/3 & 2/7 \\ 0 & 0 & 0 & 1/3 & 2/7 & 1/4 \\ 0 & 0 & 0 & 2/7 & 1/4 & 2/9 \end{bmatrix}.$$

This uses a formula from [Goo61], extended to matrix polynomials. The six generalized eigenvalues of this pencil include, for example, one near to $-0.6854 + 1.909i$. Substituting this eigenvalue in for x in C yields a three-by-three matrix with ratio of smallest to largest singular values $\sigma_3/\sigma_1 \approx 1.7 \cdot 10^{-15}$. This is effectively singular and, thus, we have found the solutions to the nonlinear eigenvalue problem. Again note that the generalized companion matrix is not in Frobenius standard form, and that this process works for a variety of bases, including the Lagrange basis.

Circulant matrices and Vandermonde matrices.

```
> A := Matrix( 3, 3, shape=Circulant[a,b,c] );
```

$$\begin{bmatrix} a & b & c \\ c & a & b \\ b & c & a \end{bmatrix},$$

```
> F3 := Matrix( 3,3,shape=Vandermonde[ [1,exp(2*Pi*I/3),exp(4*Pi*I/3)] ] );
```

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1/2 + 1/2 i \sqrt{3} & (-1/2 + 1/2 i \sqrt{3})^2 \\ 1 & -1/2 - 1/2 i \sqrt{3} & (-1/2 - 1/2 i \sqrt{3})^2 \end{bmatrix}.$$

It is easy to see that the F3 matrix diagonalizes the circulant matrix A.

Toeplitz and Hankel matrices. These can be constructed by calling `ToeplitzMatrix` and `HankelMatrix`, or by direct use of the `shape` option of the `Matrix` constructor.

```
> T := ToeplitzMatrix( [a,b,c,d,e,f,g] );
```

```
> T := Matrix( 4,4,shape=Toeplitz[false,Vector(7,[a,b,c,d,e,f,g])] );
```

both yield a matrix that looks like

$$\begin{bmatrix} d & c & b & a \\ e & d & c & b \\ f & e & d & c \\ g & f & e & d \end{bmatrix},$$

though in the second case only 7 storage locations are used, whereas 16 are used in the first. This economy may be useful for larger matrices. The `shape` constructor for `Toeplitz` also takes a Boolean argument `true`, meaning symmetric.

Both `Hankel` and `Toeplitz` matrices may be specified with an indexed symbol for the entries:

```
> H := Matrix( 4, 4, shape=Hankel[a] ); yields
```

$$\begin{bmatrix} a_1 & a_2 & a_3 & a_4 \\ a_2 & a_3 & a_4 & a_5 \\ a_3 & a_4 & a_5 & a_6 \\ a_4 & a_5 & a_6 & a_7 \end{bmatrix}.$$

72.11 Functions of Matrices

The exponential of the matrix A is computed in the `MatrixExponential` command of Maple by *polynomial interpolation* (see Chapter 11.1) of the exponential at each of the eigenvalues of A , including multiplicities. In an exact computation context, this method is not so “dubious” [Lab97]. This approach is also used by the general `MatrixFunction` command.

Examples:

```
> A := Matrix( 3, 3, [[-7,-4,-3],[10,6,4],[6,3,3]] );
> MatrixExponential( A );
```

$$\begin{bmatrix} 6-7e^1 & 3-4e^1 & 2-3e^1 \\ 10e^1-6 & -3+6e^1 & -2+4e^1 \\ 6e^1-6 & -3+3e^1 & -2+3e^1 \end{bmatrix} \quad (72.1)$$

Now a square root: `> MatrixFunction( A, sqrt(x), x ):`

$$\begin{bmatrix} -6 & -7/2 & -5/2 \\ 8 & 5 & 3 \\ 6 & 3 & 3 \end{bmatrix} \quad (72.2)$$

Another matrix square root example, for a matrix close to one that has no square root:

```
> A := Matrix( 2, 2, [[epsilon^2, 1], [0, delta^2]] );
> S := MatrixFunction( A, sqrt(x), x );
> simplify( S ) assuming positive;
```

$$\begin{bmatrix} \epsilon & \frac{1}{\epsilon + \delta} \\ 0 & \delta \end{bmatrix} \quad (72.3)$$

If ϵ and δ both approach zero, we see that the square root has an entry that approaches infinity. Calling `MatrixFunction` on the above matrix with $\epsilon = \delta = 0$ yields an error message, `Matrix function x^(1/2) is not defined for this Matrix`, which is correct.

Now for the matrix logarithm.

```
> Pascal := Matrix( 4, 4, (i,j)->binomial(j-1,i-1) );
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (72.4)$$

```
> MatrixFunction( Pascal, log(x), x );
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (72.5)$$

Now a function not covered in Chapter 11, instead of redoing the sine and cosine examples:

```
> A := Matrix( 2, 2, [[-1/5, 1], [0, -1/5]] );
```

```
> W := MatrixFunction( A, LambertW(-1,x), x );
```

$$W := \begin{bmatrix} \text{LambertW}(-1, -1/5) & -5 \frac{\text{LambertW}(-1, -1/5)}{1 + \text{LambertW}(-1, -1/5)} \\ 0 & \text{LambertW}(-1, -1/5) \end{bmatrix} \quad (72.6)$$

```
> evalf( W );
```

$$\begin{bmatrix} -2.542641358 & -8.241194055 \\ 0.0 & -2.542641358 \end{bmatrix} \quad (72.7)$$

That matrix satisfies $W \exp(W) = A$, and is a *primary matrix function*. (See [CGH⁺96] for more details about the Lambert W function.)

Now the matrix sign function (cf. Chapter 11.6). Consider

```
> Pascal2 := Matrix( 4, 4, (i,j)->(-1)^(i-1)*binomial(j-1,i-1) );
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -1 & -2 & -3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & -1 \end{bmatrix}. \quad (72.8)$$

Then we compute the matrix sign function of this matrix by

```
> S := MatrixFunction( Pascal2, csgn(z), z );
```

which turns out to be the same matrix (Pascal2).

Note: The complex “sign” function we use here is not the usual complex sign function for scalars

$$\text{signum}(re^{i\theta}) := e^{i\theta},$$

but rather (as desired for the definition of the matrix sign function)

$$\text{csgn}(z) = \begin{cases} 1 & \text{if } \text{Re}(z) > 0 \\ -1 & \text{if } \text{Re}(z) < 0 \\ \text{signum}(\text{Im}(z)) & \text{if } \text{Re}(z) = 0 \end{cases}.$$

This has the side effect of making the function defined even when the input matrix has purely imaginary eigenvalues. The `signum` and `csgn` of 0 are both 0, by default, but can be specified differently if desired.

Cautions:

1. Further, it is not the `sign` function in MAPLE, which is a different function entirely: That function (`sign`) returns the sign of the leading coefficient of the polynomial input to `sign`.
2. (In General) This general approach to computing matrix functions can be slow for large exact or symbolic matrices (because manipulation of symbolic representations of the eigenvalues using `RootOf`, typically encountered for $n \geq 5$, can be expensive), and on the other hand can be unstable for floating-point matrices, as is well known, especially those with nearly multiple eigenvalues. However, for small or for structured matrices this approach can be very useful and can give insight.

72.12 Matrix Stability

As defined in Chapter 19, a matrix is (negative) *stable* if all its eigenvalues are in the left half plane (in this section, “stable” means “negative stable”). In Maple, one may test this by direct computation of the eigenvalues (if the entries of the matrix are numeric) and this is likely faster and more accurate than any purely rational operation based test such as the Hurwitz criterion. If, however, the matrix contains symbolic entries, then one usually wishes to know for what values of the parameters the matrix is stable. We may obtain conditions on these parameters by using the Hurwitz command of the `PolynomialTools` package on the characteristic polynomial.

Examples:

Negative of `gallery(3)` from MATLAB.

```
> A := -Matrix( [[-149, -50, -154], [537, 180, 546], [-27, -9, -25]] ):
> E := Matrix( [[130, -390, 0], [43, -129, 0], [133, -399, 0]] ):
> AtE := A - t*E;
```

$$\begin{bmatrix} 149 - 130t & 50 + 390t & 154 \\ -537 - 43t & -180 + 129t & -546 \\ 27 - 133t & 9 + 399t & 25 \end{bmatrix} \quad (72.9)$$

For which t is that matrix stable?

```
> p := CharacteristicPolynomial( AtE, lambda );
> PolynomialTools[Hurwitz]( p, lambda, 's', 'g' );
```

This command returns “FAIL,” meaning that it cannot tell whether p is stable or not; this is only to be expected as t has not yet been specified. However, according to the documentation, all coefficients of λ returned in s must be positive, in order for p to be stable. The coefficients returned are

$$\left[\frac{\lambda}{6+t}, 1/4 \frac{(6+t)^2 \lambda}{15 + 433453t + 123128t^2}, \frac{(60 + 1733812t + 492512t^2) \lambda}{(6+t)(6 + 1221271t)} \right] \quad (72.10)$$

and analysis (not given here) shows that these are all positive if and only if $t > -6/1221271$.

Acknowledgements

Many people have contributed to linear algebra in Maple, for many years. Dave Hare and David Linder deserve particular credit, especially for the LinearAlgebra package and its connections to CLAPACK, and have also greatly helped our understanding of the best way to use this package. We are grateful to Dave Linder and to Jürgen Gerhard for comments on early drafts of this chapter.

References

- [CGH⁺96] Robert M. Corless, Gaston H. Gonnet, D.E.G. Hare, David J. Jeffrey, and Donald E. Knuth. On the Lambert W function. *Adv. Comp. Math.*, 5:329–359, 1996.
- [CJ92] R.M. Corless and D.J. Jeffrey. Well... it isn't quite that simple. *SIGSAM Bull.*, 26(3):2–6, 1992.
- [CJ97] R.M. Corless and D.J. Jeffrey. The Turing factorization of a rectangular matrix. *SIGSAM Bull.*, 31(3):20–28, 1997.
- [Cor93] Robert M. Corless. Six, lies, and calculators. *Am. Math. Month.*, 100(4):344–350, April 1993.
- [Cor04] Robert M. Corless. Generalized companion matrices in the Lagrange basis. In Laureano Gonzalez-Vega and Tomas Recio, Eds., *Proceedings EACA*, pp. 317–322, June 2004.
- [CW04] Robert M. Corless and Stephen M. Watt. Bernstein bases are optimal, but, sometimes, Lagrange bases are better. In *Proceedings of SYNASC*, pp. 141–153. Mirton Press, September 2004.
- [Goo61] I.J. Good. The colleague matrix, a Chebyshev analogue of the companion matrix. *Q. J. Math.*, 12:61–68, 1961.
- [JV05] Gudbjorn F. Jónsson and Steven Vavasis. Solving polynomials with small leading coefficients. *Siam J. Matrix Anal. Appl.*, 26(2):400–414, 2005.
- [Lab97] George Labahn, Personal Communication, 1997.
- [Win04] Joab R. Winkler. The transformation of the companion matrix resultant between the power and Bernstein polynomial bases. *Appl. Num. Math.*, 48(1):113–126, 2004.

73

Mathematica

73.1	Introduction	73-1
73.2	Vectors	73-2
73.3	Basics of Matrices	73-5
73.4	Matrix Algebra	73-9
73.5	Manipulation of Matrices	73-13
73.6	Eigenvalues	73-14
73.7	Singular Values	73-16
73.8	Decompositions	73-18
73.9	Linear Systems	73-20
73.10	Linear Programming	73-23
	Appendix	73-25
	References	73-27

Heikki Ruskeepää
University of Turku

73.1 Introduction

About *Mathematica*[®]

Mathematica is a comprehensive software system for doing symbolic and numerical calculations, creating graphics and animations, writing programs, and preparing documents.

The heart of *Mathematica* is its broad collection of tools for symbolic and exact mathematics, but numerical methods also form an essential part of the system. *Mathematica* is known for its high-quality graphics, and the system is also a powerful programming language, supporting both traditional procedural techniques and functional and rule-based programming. In addition, *Mathematica* is an environment for preparing high-quality documents.

The first version of *Mathematica* was released in 1988. The current version, version 5, was released in 2003. *Mathematica* now contains over 4000 commands and is one of the largest single application programs ever developed. *Mathematica* is a product of Wolfram Research, Inc. The founder, president, and CEO of Wolfram Research is Stephen Wolfram.

Mathematica contains two main parts — the **kernel** and the **front end**. The kernel does the computations. For example, the implementation of the **Integrate** command comprises about 500 pages of *Mathematica* code and 600 pages of C code. The front end is a user interface that takes care of the communication between the user and the kernel. In addition, there are **packages** that supplement the kernel; packages have to be loaded as needed.

The most common type of user interface is based on interactive documents known as **notebooks**. *Mathematica* is often used like an advanced calculator for a moment's need, in which case a notebook is simply an interface to write the commands and read the results. However, often a notebook grows to be a useful document that you will save or print or use in a presentation.

Notebooks consist of **cells**. Each cell is indicated with a **cell bracket** at the right side of the notebook. Cells are grouped in various ways so as to form a hierarchical structure. For example, each input is in a cell, each output is in a cell, and each input–output pair forms a higher-level cell.

About this Chapter

Within the limited space of this chapter, we cover the essentials of doing linear algebra calculations with *Mathematica*. The chapter covers many of the topics of the handbook done via *Mathematica*, and the reader should consult the relevant section for further information.

Most commands are demonstrated in the Examples sections, but note that the examples are very simple. Indeed, the aim of these examples is only to show how the commands are used and what kind of result we get in simple cases. We do not demonstrate the full power and every feature of the commands.

Commands relating to packages are frequently mentioned in this chapter, but they are often not fully explained or demonstrated. *Mathematica* has advanced technology for sparse matrices, but we only briefly mention them in Section 73.3; for a detailed coverage, we refer to [WR03]. The basic principle is that all calculations with sparse matrices work as for usual matrices. [WR03] also considers performance and efficiency questions.

The Appendix contains a short introduction to the use of *Mathematica*. There we also refer to some books, documents, and Help Browser material where you can find further information about linear algebra with *Mathematica*.

This chapter was written with *Mathematica* 5.2. However, many users of *Mathematica* have earlier versions. To help these users, we have denoted by (Mma 5.0) and (Mma 5.1) the features of *Mathematica* that are new in versions 5.0 and 5.1, respectively. In the Appendix we list ways to do some calculations with earlier versions.

As in *Mathematica* notebooks, in this chapter *Mathematica* commands and their arguments are in boldface while outputs are in a plain font. *Mathematica* normally shows the output below the input, but here, in order to save space, we mostly show the output next to the input.

Matrices are traditionally denoted by capital letters like *A* or *M*. However, in *Mathematica* we have the general advice that all user-defined names should begin with lower-case letters so that they do not conflict with the built-in names, which always begin with an upper-case letter. We follow this advice and mainly use the lower-case letter **m** for matrices. In principle, we could use most upper-case letters, but note that the letters **C**, **D**, **E**, **I**, **N**, and **O** are reserved names in *Mathematica* and they cannot be used as user-defined names.

Because the **LinearAlgebra`MatrixManipulation`** package appears quite frequently in this chapter, we abbreviate it to **LAMM**.

73.2 Vectors

Commands:

Vectors in *Mathematica* are lists. In Section 73.3, we will see that matrices are lists of lists, each sublist being a row of the matrix. Note that *Mathematica* does not distinguish between row and column vectors. Indeed, when we compute with vectors and matrices, in most cases it is evident for *Mathematica* how an expression has to be calculated. Only in some rare cases do we need to be careful and write an expression in such a way that *Mathematica* understands the expression in the correct way. One of these cases is the multiplication of a column by a row vector; this has to be done with an outer product (see **Outer** in item 4 below).

1. Vectors in *Mathematica*:

- **{a, b, c, ...}** A vector with elements **a**, **b**, **c**, ...
- **MatrixForm[v]** Display vector **v** in a column form.

- **Length[v]** The number of elements of vector **v**.
 - **VectorQ[v]** Test whether **v** is a vector.
2. Generation of vectors:
- **Range[n₁]** Create the vector (1, 2, ..., **n₁**). With **Range[n₀, n₁]** we get the vector (**n₀**, **n₀ + 1**, ..., **n₁**) and with **Range[n₀, n₁, d]** the vector (**n₀**, **n₀ + d**, **n₀ + 2d**, ..., **n₁**).
 - **Table[expr, {i, n₁}]** Create a vector by giving **i** the values 1, 2, ..., **n₁** in **expr**. If the iteration specification is {**i**, **n₀**, **n₁**}, then **i** gets the values **n₀**, **n₀ + 1**, ..., **n₁**, and for {**i**, **n₀**, **n₁**, **d**}, the values of **i** are between **n₀** and **n₁** in steps of **d**. For {**n₁**}, simply **n₁** copies of **expr** are taken.
 - **Array[f, n₁]** Create the **n₁** vector (**f[1]**, ..., **f[n₁]**). With **Array[f, n₁, n₀]** we get the **n₁** vector (**f[n₀]**, ..., **f[n₀ + n₁ - 1]**).
3. Calculating with vectors:
- **a v** Multiply vector **v** with scalar **a**.
 - **u + v** Add two vectors.
 - **u v** Multiply the corresponding elements of two vectors and form a vector from the products (there is a space between **u** and **v**); for an inner product, write **u.v**.
 - **u/v** Divide the corresponding elements of two vectors.
 - **v^p** Calculate the **p**th power of each element of a vector.
 - **a^v** Generate a vector by calculating the powers of scalar **a** that are given in vector **v**.
4. Products of vectors:
- **u.v** The inner product of two vectors of the same size.
 - **Outer[Times, u, v]** The outer product (a matrix) of vectors **u** and **v**.
 - **Cross[u, v]** The cross product of two vectors.
5. Norms and sums of vectors:
- **Norm[v]** (Mma 5.0) The 2-norm (or Euclidean norm) of a vector.
 - **Norm[v, p]** (Mma 5.0) The **p**-norm of a vector (**p** is a number in [1, ∞) or ∞).
 - **Total[v]** (Mma 5.0) The sum of the elements of a vector.
 - **Apply[Times, v]** The product of the elements of a vector.
6. Manipulation of vectors:
- **v[[i]]** Take element **i** (output is the corresponding scalar).
 - **v[[i]] = a** Change the value of element **i** into scalar **a** (output is **a**).
 - **v[{i, j, ...}]** Take elements **i, j, ...** (output is the corresponding vector).
 - **First[v], Last[v]** Take the first/last element (output is the corresponding scalar).
 - **Rest[v], Most[v]** Drop the first/last element (output is the corresponding vector).
 - **Take[v, n], Take[v, -n], Take[v, {n₁, n₂}]** Take the first **n** elements / the last **n** elements / elements **n₁, ..., n₂** (output is the corresponding vector).
 - **Drop[v, n], Drop[v, -n], Drop[v, {n₁, n₂}]** Drop the first **n** elements / the last **n** elements / elements **n₁, ..., n₂** (output is the corresponding vector).
 - **Prepend[v, a], Append[v, a]** Insert element **a** at the beginning/end of a vector (output is the corresponding vector).
 - **Join[u, v, ...]** Join the given vectors into one vector (output is the corresponding vector).

7. In the **LinearAlgebra`Orthogonalization`** package:
 - **GramSchmidt[{u, v, ...}]** Generate an orthonormal set from the given vectors.
 - **Projection[u, v]** Calculate the orthogonal projection of **u** onto **v**.
8. In the **Geometry`Rotations`** package: rotations of vectors.

Examples:

1. Vectors in *Mathematica*.

```
v = {4, 2, 3}    {4, 2, 3}
MatrixForm[v]     $\begin{pmatrix} 4 \\ 2 \\ 3 \end{pmatrix}$ 
Length[v]      3
VectorQ[v]     True
```

2. Generation of vectors. **Range** is nice for forming lists of integers or reals:

```
Range[10]    {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
Range[0, 10, 2] {0, 2, 4, 6, 8, 10}
Range[1.5, 2, 0.1] {1.5, 1.6, 1.7, 1.8, 1.9, 2.}
```

Table is one of the most useful commands in *Mathematica*:

```
Table[Random[], {3}] {0.454447, 0.705133, 0.226419}
Table[Random[Integer, {1, 6}], {5}] {2, 3, 6, 3, 4}
Table[x^i, {i, 5}] {x, x2, x3, x4, x5}
Table[x[i], {i, 5}] {x[1], x[2], x[3], x[4], x[5]}
Table[xi, {i, 5}] {x1, x2, x3, x4, x5}
```

Array is nice for forming lists of indexed variables:

```
Array[x, 5] {x[1], x[2], x[3], x[4], x[5]}
Array[x, 5, 0] {x[0], x[1], x[2], x[3], x[4]}
```

3. Calculating with vectors. The arithmetic operations of multiplying a vector with a scalar and adding two vectors work in *Mathematica* as expected. However, note that *Mathematica* also does other types of arithmetic operations with vectors — multiplication, division, and powers. All arithmetic operations are done in an element-by-element way. For example, **u v** and **u/v** form a vector from the products or quotients of the corresponding elements of **u** and **v**. This is a useful property in many calculations, but remember to use **u.v** (which is the same as **Dot[u, v]**) for an inner product.

```
u = {a, b, c}; v = {4, 2, 3};
{10 v, u + v, u v}
{{40, 20, 30}, {4 + a, 2 + b, 3 + c}, {4 a, 2 b, 3 c}}
{1/v, u/v, u^2}  {{1/4, 1/2, 1/3}, {a/4, b/2, c/3}, {a2, b2, c2}}
```

Functions of vectors are also calculated elementwise:

```
Log[v] {Log[4], Log[2], Log[3]}
```

4. Products of vectors. With **u** and **v** as in Example 3, we calculate an inner product, an outer product, and a cross product:

```
v.u 4 a + 2 b + 3 c
Outer[Times, v, u]
{{4 a, 4 b, 4 c}, {2 a, 2 b, 2 c}, {3 a, 3 b, 3 c}}
```

```

MatrixForm[%] 
$$\begin{pmatrix} 4a & 4b & 4c \\ 2a & 2b & 2c \\ 3a & 3b & 3c \end{pmatrix}$$

Cross[v, u] {-3b + 2c, 3a - 4c, -2a + 4b}

```

5. Norms and sums of vectors. The default vector norm is the 2-norm:

```

Norm[u]  $\sqrt{\text{Abs}[a]^2 + \text{Abs}[b]^2 + \text{Abs}[c]^2}$ 
Norm[u, 2]  $\sqrt{\text{Abs}[a]^2 + \text{Abs}[b]^2 + \text{Abs}[c]^2}$ 
Norm[u, 1]  $\text{Abs}[a] + \text{Abs}[b] + \text{Abs}[c]$ 
Norm[u, ∞]  $\text{Max}[\text{Abs}[a], \text{Abs}[b], \text{Abs}[c]]$ 
Total[u]  $a + b + c$ 
Apply[Times, u]  $a \, b \, c$ 

```

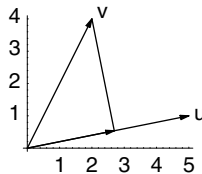
Applications:

1. (Plotting of vectors) A package [WR99, p. 133] defines the graphic primitive **Arrow** which can be used to plot vectors. As an example, we compute the orthogonal projection of a vector onto another vector and show the three vectors (for graphics primitives like **Arrow**, **Line**, and **Text**, see [Rus04, pp. 132–146]):

```

<< LinearAlgebra`Orthogonalization`
<< Graphics`Arrow`
u = {5, 1}; v = {2, 4};
pr = Projection[v, u] {35/13, 7/13}
Show[Graphics[{Arrow[{0, 0}, u], Arrow[{0, 0}, v],
  Arrow[{0, 0}, pr], Line[{v, pr}], Text["u", u, {-1, 0}],
  Text["v", v, {-1, -0.4}]}], Axes -> True,
  AspectRatio -> Automatic, PlotRange -> All];

```



73.3 Basics of Matrices

Commands:

Matrices in *Mathematica* are lists of lists, each sublist being a row of the matrix. Although the traditional symbols for matrices are capital letters like *A* or *M*, we use the lower-case letter **m** for most matrices because, in *Mathematica*, all user-defined names should begin with a lower-case letter.

Here we only briefly mention sparse matrices; a detailed exposition can be found in [WR03]; this document is the item Built-in Functions ▸ Advanced Documentation ▸ Linear Algebra in the Help Browser of *Mathematica*.

1. Matrices in *Mathematica*:

- **{{a, b, ...}, {c, d, ...}, ...}** A matrix with rows $(a, b, \dots)$, $(c, d, \dots)$, $\dots$.
- **MatrixForm[m]** Display matrix **m** in a two-dimensional form.
- **Length[m]** The number of rows of matrix **m**.
- **Dimensions[m]** The number of rows and columns of matrix **m**.
- **MatrixQ[m]** Test whether **m** is a matrix.
- The menu command Input ▸ Create Table/Matrix/Palette generates an empty matrix into which the elements can be written.

2. Generation of matrices:

- **IdentityMatrix[n]** An $n \times n$ identity matrix.
- **DiagonalMatrix[{a, b, ...}]** A diagonal matrix with diagonal elements **a, b, ...**.
- **HilbertMatrix, HankelMatrix, ZeroMatrix, UpperDiagonalMatrix, LowerDiagonalMatrix, TridiagonalMatrix** (in the **LAMM** package).
- **Table[expr, {i, m₁}, {j, n₁}]** Create a matrix by giving, in **expr**, **i** the values 1, 2, ..., **m₁** and, for each **i, j** the values 1, ..., **n₁**. Other forms of an iteration specification are **{m₁}**, **{i, m₀, m₁}**, and **{i, m₀, m₁, d}** (see item 2 in Section 73.2).
- **Array[f, {m₁, n₁}]** Create an $m_1 \times n_1$ matrix with elements **f[i, j]**.
- **Array[f, {m₁, n₁}, {m₀, n₀}]** Create an $m_1 \times n_1$ matrix using starting values **m₀** and **n₀** for the indices (the default values of **m₀** and **n₀** are 1).

3. Sparse matrices:

- **SparseArray[rules]** (Mma 5.0) Create a vector or matrix by taking nonzero elements from **rules** and setting other elements to zero.
- **Normal[s]** (Mma 5.0) Show a sparse vector or matrix **s** in the usual list form.
- **ArrayRules[m]** (Mma 5.0) Show all nonzero elements of a vector or matrix as rules.

4. Arithmetic of matrices (see also Section 73.4):

- **a m** Multiply matrix **m** by scalar **a**.
- **m + n** Add two matrices.
- **m n** Multiply *the corresponding elements* of two matrices (giving the Hadamard product of the matrices; there is a space between **m** and **n**); for a proper matrix product, write **m.n** (see item 2 in Section 73.4).
- **m^p** Calculate the **p**th power of *each element* of a matrix; for a proper matrix power, write **MatrixPower[m, p]** (see item 3 in Section 73.4).
- **m^-1** Calculate the reciprocal of *each element* of a matrix; for a proper matrix inverse, write **Inverse[m]** (see item 5 in Section 73.4).

5. Sums and trace of matrices:

- **Total[m]** (Mma 5.0) The vector of column sums (i.e., the sums of the elements of each column).
- **Total[Transpose[m]]** or **Map[Total, m]** (Mma 5.0) The vector of row sums.
- **Total[Flatten[m]]** or **Total[m, 2]** (Mma 5.0) The sum of all elements.
- **Tr[m]** Trace.
- **Tr[m, List]** List of diagonal elements.

6. Plots of matrices:

- **ArrayPlot[m]** (Mma 5.1) Plot a matrix by showing zero values as white squares, the maximum absolute value as black squares, and values between as gray squares.
- **ArrayPlot[m, ColorRules → {0 → White, _ → Black}]** (Mma 5.1) Show all nonzero values as black squares (also **MatrixPlot[m]** (Mma 5.0) from the **LAMM** package).

Examples:

1. Matrices in *Mathematica*. Matrices are lists of rows:

```
m = {{5, 2, 3}, {4, 0, 2}}  {{5, 2, 3}, {4, 0, 2}}
MatrixForm[m]  
$$\begin{pmatrix} 5 & 2 & 3 \\ 4 & 0 & 2 \end{pmatrix}$$

```

The number of rows and the size of the matrix are

```
Length[m]  2
Dimensions[m]  {2, 3}
```

With the menu command Input ▸ Create Table/Matrix/Palette, we get a matrix with empty slots that can then be filled with the help of the Tab key:

```

$$\begin{pmatrix} \square & \square & \square \\ \square & \square & \square \end{pmatrix}$$

```

With the **TableSpacing** option of **MatrixForm** we can control the space between rows and columns (the default value of the option is 1):

```
MatrixForm[m, TableSpacing → {.5, .5}]  
$$\begin{pmatrix} 5 & 2 & 3 \\ 4 & 0 & 2 \end{pmatrix}$$

```

Note that *Mathematica* will not do any calculations with a matrix that is in a matrix form! For example, writing

```
m = {{5, 2, 3}, {4, 0, 2}} // MatrixForm  
$$\begin{pmatrix} 5 & 2 & 3 \\ 4 & 0 & 2 \end{pmatrix}$$

```

defines **m** to be the matrix form of the given matrix and *Mathematica* does not do any calculations with this **m**. Write instead

```
MatrixForm[m = {{5, 2, 3}, {4, 0, 2}}]  
$$\begin{pmatrix} 5 & 2 & 3 \\ 4 & 0 & 2 \end{pmatrix}$$

```

or (**m = {{5, 2, 3}, {4, 0, 2}}**) // **MatrixForm** or do as we did above, define the matrix with **m = {{5, 2, 3}, {4, 0, 2}}** and then display the matrix with **MatrixForm[m]**.

2. Generation of matrices.

```

IdentityMatrix[3]  {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
DiagonalMatrix[{3, 1, 2}]  {{3, 0, 0}, {0, 1, 0}, {0, 0, 2}}
Table[Random[], {2}, {2}]
{{0.544884, 0.397472}, {0.308083, 0.191191}}
Table[1/(i + j - 1), {i, 3}, {j, 3}]
{{1, 1/2, 1/3}, {1/2, 1/3, 1/4}, {1/3, 1/4, 1/5}}

```

If, **Which**, and **Switch** are often useful in connection with **Table**. In the following examples, we form the same tridiagonal matrix in three ways (we show the result only for the first example):

```

Table[If[i == j, 13, If[i == j - 1, 11,
If[i == j + 1, 12, 0]]], {i, 4}, {j, 4}] // MatrixForm

```

$$\begin{pmatrix} 13 & 11 & 0 & 0 \\ 12 & 13 & 11 & 0 \\ 0 & 12 & 13 & 11 \\ 0 & 0 & 12 & 13 \end{pmatrix}$$

```

Table[Which[i == j, 13, i == j - 1, 11, i == j + 1, 12, True,
0], {i, 4}, {j, 4}];

```

```

Table[Switch[i - j, -1, 11, 0, 13, 1, 12, _, 0],
{i, 4}, {j, 4}];

```

Here are matrices with general elements:

```

Table[f[i, j], {i, 2}, {j, 2}]
{{f[1, 1], f[1, 2]}, {f[2, 1], f[2, 2]}}
Table[fi,j, {i, 2}, {j, 2}]  {{f1,1, f1,2}, {f2,1, f2,2}}
Array[f, {2, 2}]  {{f[1, 1], f[1, 2]}, {f[2, 1], f[2, 2]}}

```

3. Sparse matrices. With **SparseArray** we can input a matrix by specifying only the nonzero elements. With **Normal** we can show a sparse matrix as a usual matrix. One way to specify the rules is to write them separately:

```

SparseArray[{{1, 1} → 3, {2, 3} → 2, {3, 2} → 4}]
SparseArray[<3>, {3, 3}]
% // Normal  {{3, 0, 0}, {0, 0, 2}, {0, 4, 0}}

```

Another way is to gather the left- and right-hand sides of the rules together into lists:

```

SparseArray[{{1, 1}, {2, 3}, {3, 2}} → {3, 2, 4}];

```

Still another way is to give the rules in the form of index patterns. As an example, we generate the same tridiagonal matrix as we generated in Example 2 with **Table**:

```

SparseArray[{{i-, i-} → 13, {i-, j-} /; i - j == -1 → 11,
{i-, j-} /; i - j == 1 → 12}, {4, 4}];

```

4. Arithmetic of matrices. The arithmetic operations of multiplying a matrix with a scalar and adding two matrices work in *Mathematica* as expected. However, note that *Mathematica* also does other types of arithmetic operations with matrices, especially multiplication and powers. All arithmetic operations are done in an element-by-element way. For example, **m n** forms a matrix from the products of the corresponding elements of **m** and **n**. This is a useful property in some calculations, but remember to use **m.n** (which is the same as **Dot**[**m**, **n**]) for an inner product, **MatrixPower**[**m**, **p**] for a matrix power, and **Inverse**[**m**] for a matrix inverse (see Section 73.4).

```

m = {{2, 5}, {3, 1}};
n = {{a, b}, {c, d}};

```

```

3 n {{3 a, 3 b}, {3 c, 3 d}}
m + n {{2 + a, 5 + b}, {3 + c, 1 + d}}
m n {{2 a, 5 b}, {3 c, d}}
n^2 {{a^2, b^2}, {c^2, d^2}}
n^-1 {{1/a, 1/b}, {1/c, 1/d}}

```

5. Sums and trace of matrices. With **m** as in Example 4, here are the column sums, row sums, the sum of all elements, and trace:

```

{Total[m], Total[Transpose[m]], Total[Flatten[m]], Tr[m]}
{{5, 6}, {7, 4}, 11, 3}

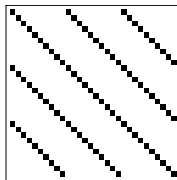
```

6. Plots of matrices.

```

ArrayPlot[{{5, 2, 3}, {4, 0, 2}}];
m = SparseArray[{{i_, j_} /; Mod[i - j, 10] == 0 -> 1}, {30, 30}];
ArrayPlot[m];

```



73.4 Matrix Algebra

Commands:

- Transpose:
 - **Transpose[m]** Transpose.
 - **ConjugateTranspose[m]** (Mma 5.1) Conjugate transpose.
- Product:
 - **m.n** Product of two matrices.
 - **m.v** Product of a matrix and a (column) vector.
 - **v.m** Product of a (row) vector and a matrix.
- Power and matrix exponential:
 - **MatrixPower[m, p]** *p*th power of a square matrix.
 - **MatrixExp[m]** Matrix exponential $e^{\mathbf{m}} = \sum_{i=0}^{\infty} \frac{1}{i!} \mathbf{m}^i$ of a square matrix.

4. Determinant and minors:

- **Det[m]** Determinant of a square matrix.
- **Minors[m]** Minors of a square matrix.
- **Minors[m, k]** k th minors.

5. Inverse and pseudo-inverse:

- **Inverse[m]** Inverse of a square matrix.
- **PseudoInverse[m]** Pseudo-inverse (of a possibly rectangular matrix).

6. Norms and condition numbers:

- **Norm[m]** (Mma 5.0) The 2-norm of a numerical matrix.
- **Norm[m, p]** (Mma 5.0) The p -norm of a matrix (p is a number in $[1, \infty)$ or ∞ or **Frobenius**).
- **InverseMatrixNorm[m, p]** The p -norm of the inverse (in the **LAMM** package).
- **MatrixConditionNumber[m, p]** The p -norm matrix condition number (in the **LAMM** package).

7. Nullspace, rank, and row reduction:

- **NullSpace[m]** Basis vectors of the null space (kernel).
- **MatrixRank[m]** (Mma 5.0) Rank.
- **RowReduce[m]** Do Gauss–Jordan elimination to produce the reduced row echelon form.

Examples:

1. Transpose.

```

m = {{2 + 3 I, 1 - 2 I}, {3 - 4 I, 4 + I}};
MatrixForm[m]       $\begin{pmatrix} 2 + 3i & 1 - 2i \\ 3 - 4i & 4 + i \end{pmatrix}$ 

MatrixForm[Transpose[m]]       $\begin{pmatrix} 2 + 3i & 3 - 4i \\ 1 - 2i & 4 + i \end{pmatrix}$ 

MatrixForm[ConjugateTranspose[m]]       $\begin{pmatrix} 2 - 3i & 3 + 4i \\ 1 + 2i & 4 - i \end{pmatrix}$ 

```

2. Product. Consider the following matrices and vectors:

```

m = {{1, 2, 3}, {4, 5, 6}}; n = {{p, q}, {r, s}, {t, u}};
v = {1, 2}; w = {1, 2, 3};
Map[MatrixForm, {m, n, v, w}]

```

$$\left\{ \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \begin{pmatrix} p & q \\ r & s \\ t & u \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \right\}$$

Here are some products:

```

m.n      {{p + 2r + 3t, q + 2s + 3u}, {4p + 5r + 6t, 4q + 5s + 6u}}
n.v      {p + 2q, r + 2s, t + 2u}
w.n      {p + 2r + 3t, q + 2s + 3u}

```

3. Power and matrix exponential. Powers can be calculated with **MatrixPower** or by using the dot repeatedly:

```

k = {{2, 3}, {-2, 1}};
MatrixPower[k, 4]  {{-50, -63}, {42, -29}}
k.k.k.k  {{-50, -63}, {42, -29}}
MatrixExp[k // N]  {{-2.6658, 3.79592}, {-2.53061, -3.93111}}

```

4. Determinant and minors. First, we consider a nonsingular matrix:

```

m = {{6, 2, 3}, {2, 7, 5}, {4, 2, 5}};
Det[m]  98
Minors[m]  {{38, 24, -11}, {4, 18, 4}, {-24, -10, 25}}

```

Here is a singular matrix:

```

n = {{7, 7, 2, 7}, {2, 5, 0, 7}, {8, 7, 8, 3}, {5, 6, 4, 5}};
Det[n]  0

```

5. Inverse and pseudo-inverse. First, we consider the nonsingular matrix **m** from Example 4:

```

i = Inverse[m] {{25/98, -2/49, -11/98}, {5/49, 9/49, -12/49}, {-12/49, -2/49, 19/49}}
m.i-IdentityMatrix[3]  {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}

```

Then, we consider the singular matrix **n**. The inverse does not exist, but the Moore–Penrose pseudo-inverse does exist:

```

Inverse[n]
Inverse::sing :
Matrix {{7, 7, 2, 7}, {2, 5, 0, 7}, {8, 7, 8, 3}, {5, 6, 4, 5}} is
singular. More...
Inverse[{{7, 7, 2, 7}, {2, 5, 0, 7}, {8, 7, 8, 3},
{5, 6, 4, 5}}]
ps = PseudoInverse[n // N]
{{0.324527, -0.235646, -0.0236215, -0.129634},
{-0.054885, 0.0700676, 0.020876, 0.0454718},
{-0.256434, 0.102381, 0.121511, 0.111946},
{-0.0535186, 0.136327, -0.031972, 0.0521774}}

```

Next, we check the four characterizing properties of a pseudo-inverse:

```

{n.ps.n - n, ps.n.ps - ps, ConjugateTranspose[n.ps] - n.ps,
ConjugateTranspose[ps.n] - ps.n} // Chop
{{{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
{{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
{{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
{{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}}

```

For the option **Tolerance** of **PseudoInverse**, see **SingularValueList** in section 73.7.

6. Norms and condition numbers. The default matrix norm is the 2-norm (the largest singular value):

```

Norm[{{3, 1}, {4, 2}}]   $\sqrt{15 + \sqrt{221}}$ 
MatrixForm[m = {{p, q}, {r, s}}]   $\begin{pmatrix} p & q \\ r & s \end{pmatrix}$ 
Norm[m, Frobenius]   $\sqrt{\text{Abs}[p]^2 + \text{Abs}[q]^2 + \text{Abs}[r]^2 + \text{Abs}[s]^2}$ 
Norm[m, 1]  Max[Abs[p] + Abs[r], Abs[q] + Abs[s]]
Norm[m,  $\infty$ ]  Max[Abs[p] + Abs[q], Abs[r] + Abs[s]]

```

7. Nullspace, rank, and row reduction. The null space (or kernel) of a matrix M consists of vectors v for which $Mv = 0$. **NullSpace** gives a list of basis vectors for the null space. The matrix multiplied with any linear combination of these basis vectors gives a zero vector. For a nonsingular matrix M , the null space is empty (*Mathematica* gives the empty list $\{\}$), that is, there is no nonzero vector v such that $Mv = 0$. The null space of the following matrix contains two basis vectors:

```
p = {{7, 4, 8, 0}, {7, 4, 8, 0}, {1, 8, 3, 3}};
ns = NullSpace[p] {{12, -21, 0, 52}, {-4, -1, 4, 0}}
```

The matrix **p** multiplied with any linear combination of the basis vectors gives a null vector:

```
p.(a ns[[1]] + b ns[[2]]) // Simplify {0, 0, 0}
```

Matrix **p** is rank deficient:

```
MatrixRank[p] 2
```

The number of basis vectors of the null space plus the rank is the number of columns:

```
Length[NullSpace[p]] + MatrixRank[p] 4
```

The reduced row echelon form of a nonsingular square matrix is the identity matrix. Here is the reduced row echelon form of **p**:

```
RowReduce[p] // MatrixForm  $\begin{pmatrix} 1 & 0 & 1 & -\frac{3}{13} \\ 0 & 1 & \frac{1}{4} & \frac{21}{52} \\ 0 & 0 & 0 & 0 \end{pmatrix}$ 
```

The number of nonzero rows in the reduced row echelon form is the rank. **RowReduce** is also considered in item 6 of Section 73.9.

Applications:

- (Prediction of a Markov chain) We classify days to be either wet or dry and assume that weather follows a Markov chain having the states wet and dry [Rus04, pp. 730–735]. From the weather statistics of Snoqualmie Falls in Western Washington, we have estimated the transition probability matrix to be $P = \begin{pmatrix} 0.834 & 0.166 \\ 0.398 & 0.602 \end{pmatrix}$. This means that if today is a wet day, then tomorrow will be a wet day with probability 0.834 and a dry day with probability 0.166, and if today is a dry day, tomorrow will be a wet day with probability 0.398 and a dry day with probability 0.602. (See Application 1 of Section 4.3, or Chapter 54 for more information on Markov chains.)

If $\mu(n)$ is a row vector containing the probabilities of the states for day n , then $\mu(n) = \mu(0)P^n$. Assume that today is Monday and we have a dry day, then $\mu(0) = (0, 1)$. We predict the weather for Friday:

```
p = {{0.834, 0.166}, {0.398, 0.602}};
{0, 1}.MatrixPower[p, 4] {0.680173, 0.319827}
```

Thus, Friday is wet with probability 0.68 and dry with probability 0.32.

- (Permanent) (See Chapter 31 for the definition and more information about permanents.) A permanent of a square matrix M with n rows can be computed as the coefficient of the product $x_1x_2\cdots x_n$ in the product of the components of the vector Mv , where $v = (x_1, \dots, x_n)$. So, a permanent can be calculated with the following program:

```
permanent[m] := With[{v = Array[x, Length[m]]},
  Coefficient[Apply[Times, m.v], Apply[Times, v]]
m = {{6, 2, 3}, {2, 7, 5}, {4, 2, 5}};
permanent[m] 426
```

73.5 Manipulation of Matrices

Commands:

1. Taking and resetting elements:
 - **m[[i, j]]** Take element (i, j) (output is the corresponding scalar).
 - **m[[i, j]] = a** Change the value of element (i, j) into scalar a (output is a).
2. Taking, dropping, and inserting rows:
 - **m[[i]]** Take row i (output is the corresponding vector).
 - **m[[{i₁, i₂, ...}]]** Take the given rows (output is the corresponding matrix).
 - **First[m], Last[m]** Take the first/last row (output is the corresponding vector).
 - **Rest[m], Most[m]** Drop the first/last row (output is the corresponding matrix).
 - **Take[m, n], Take[m, -n], Take[m, {n₁, n₂}]** Take the first n rows / the last n rows / rows n₁, ..., n₂ (output is the corresponding matrix).
 - **Drop[m, n], Drop[m, -n], Drop[m, {n₁, n₂}]** Drop the first n rows / the last n rows / rows n₁, ..., n₂ (output is the corresponding matrix).
 - **Prepend[m, r], Append[m, r]** Insert row r at the beginning/end of a matrix (output is the corresponding matrix).
 - **Insert[m, r, i]** Insert row r between rows i - 1 and i (so, r becomes row i) (output is the corresponding matrix).
3. Taking and dropping columns:
 - **m[[All, j]]** Take column j (output is the corresponding vector).
 - **m[[All, {j₁, j₂, ...}]]** Take the given columns (output is the corresponding matrix).
 - **Take[m, All, n], Drop[m, {}, n]** Take/drop the first n columns (output is the corresponding matrix).
4. Taking submatrices:
 - **m[[{i₁, i₂, ...}, {j₁, j₂, ...}]]** Take the elements having the given indices.
 - **Take[m, {i₁, i₂}, {j₁, j₂}]** Take the elements having row indices between i₁ and i₂ and column indices between j₁ and j₂.
5. Combining and extending matrices:
 - **Join[m₁, m₂, ...]** Put matrices m_i one below the other.
 - **MapThread[Join, {m₁, m₂, ...}]** Put matrices m_i side by side; another way: **Transpose[Join[Transpose[m₁], Transpose[m₂], ...]]**.
 - **PadRight[m, {n₁, n₂}]** Extend m with zeros to an n₁ × n₂ matrix.
 - **PadRight[m, {n₁, n₂}, a]** Extend m with replicates of matrix a to an n₁ × n₂ matrix.
6. In the **LAMM** package:
 - Taking parts of matrices: **TakeRows, TakeColumns, TakeMatrix, SubMatrix**.
 - Combining matrices: **AppendColumns, AppendRows, BlockMatrix**.
7. Permuting rows and columns:
 - **m[[{i₁, ..., i_n}]]** Permute the rows into the given order.
 - **RotateLeft[m], RotateRight[m]** Move the first/last row to be the last/first row.
 - **RotateLeft[m, {0, 1}], RotateRight[m, {0, 1}]** Move the first/last column to be the last/first column.

8. Flattening and partitioning:

- **Flatten[m]** Flatten out a matrix into a vector by concatenating the rows.
- **Partition[v, k]** Partition a vector into a matrix having **k** columns and as many rows as become complete.

73.6 Eigenvalues

Commands:

1. Eigenvalues and eigenvectors:

- **CharacteristicPolynomial[m, x]** Characteristic polynomial of a square matrix [note that *Mathematica* defines the characteristic polynomial of a matrix M to be $\det(M - xI)$ while the handbook uses the definition $\det(xI - M)$].
- **Eigenvalues[m]** Eigenvalues of a square matrix, in order of decreasing absolute value (repeated eigenvalues appear with their appropriate multiplicity).
- **Eigenvectors[m]** Eigenvectors of a square matrix, in order of decreasing absolute value of their eigenvalues (eigenvectors are not normalized).
- **Eigensystem[m]** Eigenvalues and eigenvectors of a square matrix.

2. Options:

- **Eigenvalues**, **Eigenvectors**, and **Eigensystem** have the options **Cubics** (Mma 5.0) and **Quartics** (Mma 5.0) with default value **False**; if you want explicit radicals for all cubics and quartics, set **Cubics** $\rightarrow$ **True** and **Quartics** $\rightarrow$ **True**.

3. Largest eigenvalues and generalized eigenvalues:

- **Eigenvalues[m, k]** (Mma 5.0) Calculate the **k** largest eigenvalues; similarly for **Eigenvectors** and **Eigensystem**.
- **Eigenvalues[{m, a}]** (Mma 5.0) Calculate the generalized eigenvalues of matrix **m** with respect to matrix **a**; similarly for **Eigenvectors** and **Eigensystem**.

Examples:

1. Eigenvalues. First, we calculate the characteristic polynomial of **m** in two ways:

```
m = {{6, 2, 3}, {2, 7, 5}, {4, 2, 5}};
chp = CharacteristicPolynomial[m, x] 98 - 81 x + 18 x^2 - x^3
Det[m - x IdentityMatrix[3]] 98 - 81 x + 18 x^2 - x^3
```

Then, we calculate the eigenvalues in two ways:

```
lam = Eigenvalues[m] {8 + sqrt[15], 8 - sqrt[15], 2}
Solve[chp == 0] {{x -> 2}, {x -> 8 - sqrt[15]}, {x -> 8 + sqrt[15]}}
```

Here are the eigenvectors:

```
vec = Eigenvectors[m]
{{1, -9/2 + 1/2(8 + sqrt[15]), 1}, {1, -9/2 + 1/2(8 - sqrt[15]), 1},
{-5, -14, 16}}
```

An eigenvalue λ of a matrix M and the corresponding eigenvector x should satisfy $Mx = \lambda x$. We check this for the first eigenvalue and then for all the eigenvalues:

```

m.vec[[1]] - lam[[1]] vec[[1]] // Simplify {0, 0, 0}
m.Transpose[vec] - Transpose[vec].DiagonalMatrix[lam] //
Simplify
{{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}

```

With **Eigensystem** we can compute both the eigenvalues and the eigenvectors. The result is a list where the first element is a list of eigenvalues and the second element a list of the corresponding eigenvectors:

```

{lam, vec} = Eigensystem[m]
{{8 +  $\sqrt{15}$ , 8 -  $\sqrt{15}$ , 2},
 {1,  $-\frac{9}{2} + \frac{1}{2}(8 + \sqrt{15})$ , 1}, {1,  $-\frac{9}{2} + \frac{1}{2}(8 - \sqrt{15})$ , 1},
 {-5, -14, 16}}

```

- Options. For more complex eigenvalues, we may get only representations of the values as some root objects, but with **N** we get numerical values and with the **Cubics** and **Quartics** options we may get explicit eigenvalues. Note that often we only need the decimal values of the eigenvalues (not their exact expressions), and then we can input the matrix **m // N**.

```

n = {{7, 7, 2, 7}, {2, 5, 0, 7}, {8, 7, 8, 3}, {5, 6, 4, 5}};
Eigenvalues[m]
{Root[-171 + 112 #1 - 25 #12 + #13 &, 1],
 Root[-171 + 112 #1 - 25 #12 + #13 &, 3],
 Root[-171 + 112 #1 - 25 #12 + #13 &, 2], 0}
% // N {19.7731, 2.61345 + 1.34834 i, 2.61345 - 1.34834 i, 0.}
Eigenvalues[n // N]
{19.7731, 2.61345 + 1.34834 i, 2.61345 - 1.34834 i,
 -2.85924 × 10-16}
Eigenvalues[n, Cubics → True][[1]]
 $\frac{25}{3} + \frac{1}{3} \left( \frac{10667}{2} - \frac{261\sqrt{253}}{2} \right)^{1/3} + \frac{1}{3} \left( \frac{1}{2}(10667 + 261\sqrt{253}) \right)^{1/3}$ 

```

Applications:

- (Positive and negative definite matrices) A symmetric (or Hermitian) matrix is positive [negative] definite if and only if all of the eigenvalues are positive [negative]. (See Section 8.4 for more information about positive definite matrices.) With this result we can easily test the definiteness of a matrix (note that *Mathematica* does not have a special command for this test). For example, the following symmetric matrix is positive definite:

```

m = {{2, 0, 1}, {0, 4, 2}, {1, 2, 3}};
Eigenvalues[m] // N {5.66908, 2.47602, 0.854897}

```

- (A sufficient condition for a minimum or maximum) A sufficient condition for a local minimum [maximum] point of a function is that the Hessian is positive [negative] definite [BSS93, p. 134]. Calculate the stationary points of a function (for a program for classical optimization with constraints, see [Rus04, pp. 543–547]):

```

f = x^3 + x^2 y^2 + 2 x y - 3 x - y
-3 x + x^3 - y + 2 x y + x^2 y^2
grad = {D[f, x], D[f, y]}
{-3 + 3 x^2 + 2 y + 2 x y^2, -1 + 2 x + 2 x^2 y}
stat = Solve[grad == 0, {x, y}] // N
{{y → 1.22959, x → -1.16294},
 {y → 0.956992, x → 0.369407}, {y → -0.497601, x → 1.07442},
 {y → -0.844491 - 2.24741 i, x → -0.140442 - 0.584262 i},
 {y → -0.844491 + 2.24741 i, x → -0.140442 + 0.584262 i}}

```


Pick the real points (see Section 73.5, the second command in item 2 of Commands):

```
stat2 = stat[{{1, 2, 3}}]
{{y → 1.22959, x → -1.16294},
 {y → 0.956992, x → 0.369407}, {y → -0.497601, x → 1.07442}}
```

Calculate the Hessian and its eigenvalues at the real stationary points:

```
hes = {{D[f, x, x], D[f, x, y]}, {D[f, y, x], D[f, y, y]}}
{{6 x + 2 y^2, 2 + 4 x y}, {2 + 4 x y, 2 x^2}}
Map[Eigenvalues[hes /. #] &, stat2]
{{-5.61664, 4.36764}, {6.06166, -1.74063}, {6.94587, 2.30462}}
```

Thus, the first two points are saddle points while the third point is a point of minimum. Note that the gradient and the Hessian can in *Mathematica* 5.1 also be calculated as follows:

```
grad = D[f, {{x, y}, 1}]
{-3 + 3 x^2 + 2 y + 2 x y^2, -1 + 2 x + 2 x^2 y}
hes = D[f, {{x, y}, 2}]
{{6 x + 2 y^2, 2 + 4 x y}, {2 + 4 x y, 2 x^2}}
```

3. (Eigenvector–eigenvalue decomposition) Let $\text{diag}(\lambda)$ be a diagonal matrix whose diagonal elements are the eigenvalues of a matrix M , and let P be a matrix whose columns are the eigenvectors of M . If P is nonsingular, we have the decomposition $M = P\text{diag}(\lambda)P^{-1}$ or diagonalization $P^{-1}MP = \text{diag}(\lambda)$. The former equality shows that M and $\text{diag}(\lambda)$ are similar matrices, so that, for example, they have the same determinant, trace, and rank. (See Section 4.3 for more information about diagonalization.) As an example of the eigenvector–eigenvalue decomposition, first compute the eigenvalues and eigenvectors:

```
m = {{6, 2, 3}, {2, 7, 5}, {4, 2, 5}};
{lam, vec} = Eigensystem[m]
{{8 + √15, 8 - √15, 2},
 {{1, -9/2 + 1/2 (8 + √15), 1}, {1, -9/2 + 1/2 (8 - √15), 1},
 {-5, -14, 16}}}
```

Then, define the diagonal matrix of eigenvalues and the matrix whose columns are the eigenvectors:

```
diag = DiagonalMatrix[lam]
{{8 + √15, 0, 0}, {0, 8 - √15, 0}, {0, 0, 2}}
p = Transpose[vec]
{{1, 1, -5}, {-9/2 + 1/2 (8 + √15), -9/2 + 1/2 (8 - √15), -14},
 {1, 1, 16}}
```

The following computation verifies the decomposition:

```
p.diag.Inverse[p] - m // Simplify
{{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}
```

73.7 Singular Values

Commands:

1. Singular values:

- **SingularValueList[m]** (Mma 5.0) Nonzero singular values of a (possibly rectangular) numerical matrix with at least one entry that has a decimal point; the singular values are given in order of decreasing value (repeated singular values appear with their appropriate multiplicity).

2. Singular value decomposition:

- **SingularValueDecomposition[m]** (Mma 5.0) Singular value decomposition of a (possibly rectangular) numerical matrix with at least one entry that has a decimal point. For matrix M , the decomposition is $M = UWV^*$ where U and V are orthonormal and W is diagonal with singular values as the diagonal elements (* means conjugate transpose). Output is $\{U, W, V\}$.

3. An option:

- The default value **Automatic** of the option **Tolerance** (Mma 5.0) keeps only singular values larger than 100×10^{-p} , where p is **Precision[m]**. **Tolerance** $\rightarrow$ **t** keeps only singular values that are at least **t** times the largest singular value. **Tolerance** $\rightarrow$ **0** returns all singular values.

4. Largest singular values and generalized singular values:

- **SingularValueList[m, k]** (Mma 5.0) Calculate the **k** largest singular values.
- **SingularValueList[{m, a}]** (Mma 5.0) Calculate the generalized singular values of **m** with respect to **a**.

Examples:

1. Singular values. The singular values of a matrix M are the square roots of the eigenvalues of the matrix M^*M :

```
m = {{6, 2, 3}, {2, 7, 5}, {4, 2, 5}};
SingularValueList[m // N] {12.1048, 4.75064, 1.70418}
Sqrt[Eigenvalues[ConjugateTranspose[m].m]] // N
{12.1048, 4.75064, 1.70418}
```

Only nonzero singular values are returned by **SingularValueList**:

```
n = {{7, 7, 2, 7}, {2, 5, 0, 7}, {8, 7, 8, 3}, {5, 6, 4, 5}};
SingularValueList[n // N] {21.5225, 7.09914, 1.84001}
```

By giving **Tolerance** the value 0, we get all singular values:

```
SingularValueList[n // N, Tolerance  $\rightarrow$  0]
{21.5225, 7.09914, 1.84001,  $5.46569 \times 10^{-17}$ }
```

2. Singular value decomposition. Here is an example of this decomposition:

```
Map[MatrixForm, {u, w, v} = SingularValueDecomposition[m // N]]
{

$$\begin{pmatrix} -0.516443 & 0.629389 & -0.580652 \\ -0.672542 & -0.717852 & -0.179933 \\ -0.53007 & 0.297588 & 0.79402 \end{pmatrix},$$


$$\begin{pmatrix} 12.1048 & 0. & 0. \\ 0. & 4.75064 & 0. \\ 0. & 0. & 1.70418 \end{pmatrix},$$


$$\begin{pmatrix} -0.542264 & 0.743264 & -0.391801 \\ -0.561826 & -0.667491 & -0.488678 \\ -0.624741 & -0.0448685 & 0.779542 \end{pmatrix} \}$$

```

The following verifies the decomposition:

```
m = u.w.ConjugateTranspose[v] // Chop
      {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}
u.ConjugateTranspose[u] // Chop
      {{1., 0, 0}, {0, 1., 0}, {0, 0, 1.}}
v.ConjugateTranspose[v] // Chop
      {{1., 0, 0}, {0, 1., 0}, {0, 0, 1.}}
```

Applications:

1. (Condition numbers) Relative to a matrix norm $\|M\|$, the condition number of an invertible matrix M is $\|M\|\|M^{-1}\|$. (See Chapter 37 for more information about condition numbers.) A condition number is ≥ 1 ; a large condition number indicates sensitivity to round-off errors. The 2-norm condition number can be shown to be the maximum singular value divided by the minimum singular value:

```
cond[m] := With[{s = SingularValueList[m // N,  
               Tolerance → 0]}, First[s]/Last[s]]
```

For example, consider the condition numbers of Hilbert matrices:

```
h[n] := Table[1/(i + j - 1), {i, n}, {j, n}]
Table[cond[h[i]], {i, 1, 6}]
{1., 19.2815, 524.057, 15513.7, 476607., 1.49511 × 107}
```

The condition numbers grow rapidly and they are large even for small Hilbert matrices. This indicates numerical difficulties associated with Hilbert matrices. Note that the **LAMM** package defines both **HilbertMatrix** and **MatrixConditionNumber**.

73.8 Decompositions

Commands:

1. Decompositions into triangular matrices:
 - **LUdecomposition[m]** PLU decomposition of a square matrix. For matrix M , the decomposition is $PM = LU$, where L is unit lower triangular (with ones on the diagonal), U is upper triangular, and P is a permutation matrix. Output is $\{K, p, c\}$, where matrix K contains both L and U , p is a permutation vector, that is, a list specifying rows used for pivoting, and c is the L^∞ condition number of M (however, for exact matrices with no decimal points, c is 1).
 - **CholeskyDecomposition[m]** (Mma 5.0) Cholesky decomposition of a Hermitian, positive definite matrix. For matrix M , the decomposition is $M = U^*U$, where U is upper triangular. Output is U .
2. Orthogonal decompositions:
 - **SingularValueDecomposition[m]** (Mma 5.0) Singular value decomposition of a (possibly rectangular) numerical matrix with at least one entry that has a decimal point. For matrix M , the decomposition is $M = UWV^*$ where U and V are orthonormal and W is diagonal with singular values as the diagonal elements (* means conjugate transpose). Output is $\{U, W, V\}$. See item 2 in Section 73.7.
 - **QRdecomposition[m]** QR decomposition of a numerical matrix. For matrix M , the decomposition is $M = Q^*R$, where Q is orthonormal and R is upper triangular. Output is $\{Q, R\}$.
3. Decompositions related to eigenvalue problems:
 - **JordanDecomposition[m]** Jordan decomposition of a square matrix. For matrix M , the decomposition is $M = SJ S^{-1}$, where S is a similarity matrix and J is the Jordan canonical form of M . Output is $\{S, J\}$.

- **SchurDecomposition[m]** Schur decomposition of a square, numerical matrix with at least one entry that has a decimal point. For matrix M , the decomposition is $M = QTQ^*$, where Q is orthonormal and T is block upper triangular. Output is $\{Q, T\}$.
- **HessenbergDecomposition[m]** (Mma 5.1) Hessenberg decomposition of a square, numerical matrix with at least one entry that has a decimal point. For matrix M , the decomposition is $M = PHP^*$, where P is an orthonormal and H a Hessenberg matrix. Output is $\{P, H\}$.
- **PolarDecomposition[m]** Polar decomposition of a numerical matrix. For matrix M , the decomposition is $M = US$, where U is orthonormal and S is positive definite. Output is $\{U, S\}$. (In the **LAMM** package.)

Examples:

1. Decompositions into triangular matrices. As an example, we consider the PLU decomposition:

```
m = {{2., 7, 5}, {6, 2, 3}, {4, 2, 5}};
Map[MatrixForm, {lu, perm, cond} = LUDecomposition[m]]
```

$$\left\{ \begin{pmatrix} 6. & 0.333333 & 0.666667 \\ 2. & 6.33333 & 0.105263 \\ 3. & 4. & 2.57895 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix}, 9.42857 \right\}$$

The matrix **lu** contains both the L and the U matrix, **perm** is a permutation vector specifying rows used for pivoting, and **cond** is the L^∞ condition number of **m**. With a package we can extract the L and U matrices from the **lu** matrix:

```
<< LinearAlgebra`MatrixManipulation`
Map[MatrixForm, {l, u} = LUMatrices[lu]]
```

$$\left\{ \begin{pmatrix} 1. & 0. & 0. \\ 0.333333 & 1. & 0. \\ 0.666667 & 0.105263 & 1. \end{pmatrix}, \begin{pmatrix} 6. & 2. & 3. \\ 0 & 6.33333 & 4. \\ 0 & 0 & 2.57895 \end{pmatrix} \right\}$$

The following calculation verifies the PLU decomposition (see Section 73.5, the first command in item 7 of Commands):

```
m1 = m[[perm]] {{6, 2, 3}, {2., 7, 5}, {4, 2, 5}}
m1 - l.u {{0., 0., 0.}, {0., 0., 0.}, {0., 0., 0.}}
```

We can also verify the decomposition with the corresponding permutation matrix:

```
permMatr = IdentityMatrix[3][[perm]]
{{0, 1, 0}, {1, 0, 0}, {0, 0, 1}}
m2 = permMatr.m {{6., 2., 3.}, {2., 7., 5.}, {4., 2., 5.}}
m2 - l.u {{0., 0., 0.}, {0., 0., 0.}, {0., 0., 0.}}
```

Once the PLU decomposition of **m** is ready, we can solve a system of linear equations with coefficient matrix **m**. Define **b**, the constant vector, and use then **LUBackSubstitution**:

```
b = {2, 4, 1};
LUBackSubstitution[{lu, perm, cond}, b]
{0.826531, 0.530612, -0.673469}
```

However, the PLU decomposition and the back substitution are automatically done with **LinearSolve**:

```
LinearSolve[m, b] {0.826531, 0.530612, -0.673469}
```

73.9 Linear Systems

Commands:

1. Solving equations written explicitly:
 - **lhs == rhs** An equation with left-hand side **lhs** and right-hand side **rhs** (after you have written the two equal signs **==**, *Mathematica* replaces them with a corresponding single symbol).
 - **eqns = {eqn₁, eqn₂, ...}** A system of equations with the name **eqns**.
 - **Solve[eqns]** Solve the list of equations **eqns** for the symbols therein.
 - **Solve[eqns, vars]** Solve the list of equations **eqns** for the variables in list **vars**.
 - **Reduce[eqns, vars]** Give a full analysis of the solution.
2. Solving equations defined by the coefficient matrix and the constant vector:
 - **LinearSolve[m, v]** Solve the linear system **m.vars == v**.
 - **Solve[m.vars == v]** Solve the linear system **m.vars == v**.
 - If it is possible that the system has more than one solution, it is better to use **Solve**, as **LinearSolve** only gives one solution (with no warning).
3. Conversion between the two forms of linear equations:
 - **eqns = Thread[m.vars == v]** Generate explicit equations from a coefficient matrix **m** and a constant vector **v** by using the list of variables **vars**.
 - **{m, v} = LinearEquationsToMatrices[eqns, vars]** Give the coefficient matrix **m** and the constant vector **v** of given linear equations **eqns** containing variables **vars** (in the **LAMM** package).
4. Solving several systems with the same coefficient matrix:
 - **f = LinearSolve[m]** (Mma 5.0) Give a function **f** for which **f[v]** solves the system **m.x == v**.
 - **LUBackSubstitution[lu, v]** Solve the system **m.x == v**; here **lu = LUDecomposition[m]**.
5. Using the inverse and pseudo-inverse:
 - **Inverse[m].v** Give the solution of the equations **m.x == v** (this method is not recommended).
 - **PseudoInverse[m].v** Give the least-squares solution of the equations **m.x == v**.
6. Special methods:
 - **RowReduce[a]** Do Gauss–Jordan elimination to the augmented matrix **a** to produce the reduced row echelon form, where the last column is the solution of the system.
 - **TridiagonalSolve[sub, main, super, rhs]** Solve a tridiagonal system (in the **LinearAlgebra`Tridiagonal`** package).
7. Eliminating variables:
 - **Solve[eqns, vars, elims]** Solve **eqns** for **vars**, eliminating **elims**.
 - **Eliminate[eqns, elims]** Eliminate **elims** from **eqns**.

Examples:

1. Solving equations written explicitly. If the system does not have any symbols other than the variables for which the system has to be solved, we need not declare the variables in **Solve**:

```
sol = Solve[{2 x + 5 y + z == 2, 3 x + 4 y + 3 z == 2,
6 x + y + 2 z == 5}]
{{x -> 47/49, y -> 5/49, z -> -3/7}}
sol // N {{x -> 0.959184, y -> 0.102041, z -> -0.428571}}
```

Here is an example where we declare the variables:

```
eqns = {2 x + 5 y + z == 2, 3 x + 4 y + 3 z == 2,
6 x + y + 2 z == 5};
vars = {x, y, z};
sol = Solve[eqns, vars] {{x -> 47/49, y -> 5/49, z -> -3/7}}
```

Solve gives the solution in the form of transformation rules. If we want a list of the values of the variables, we can write

```
vars /. sol[[1]] {47/49, 5/49, -3/7}
```

To check the solution, write

```
eqns /. sol {{True, True, True}}
```

(Note that for an inexact solution the test may yield False for some equations due to round-off errors.) If a solution does not exist, we get an empty list:

```
Solve[{x + 2 y == 2, 2 x + 4 y == 1}] {}
```

If an infinite number of solutions exist, we get all of them (with a message):

```
Solve[{x + 2 y == 2, 2 x + 4 y == 4}] {{x -> 2 - 2 y}}
Solve::svars : Equations may not give solutions for all solve
variables. More...
```

This means that **x** has the value $2 - 2y$, while **y** can be arbitrary. The message tells that the equations may not determine the values of all of the variables (this is, indeed, the case in our example because the solution does not give the value of **y**).

The system may contain symbols:

```
eqns2 = {2 x + 5 y == 3, 3 x + d y == 2};
Solve[eqns2, {x, y}] {{x -> -10 - 3 d / (-15 + 2 d), y -> -5 / (-15 + 2 d)}}
```

Note that this is a generic solution that is valid for most values of the parameter **d**. To get a full analysis of the equations, use **Reduce**. The result is a logical statement where, for example, **&&** means logical AND and **||** means logical OR:

```
Reduce[eqns2, {x, y}]
-15 + 2 d != 0 && x == (-10 + 3 d) / (-15 + 2 d) && y == 1/5 (3 - 2 x)
```

2. Solving equations defined by the coefficient matrix and the constant vector. We continue with the same system that we considered in Example 1:

```
m = {{2, 5, 1}, {3, 4, 3}, {6, 1, 2}}; v = {2, 2, 5};
sol = LinearSolve[m, v] {47/49, 5/49, -3/7}
sol // N {0.959184, 0.102041, -0.428571}
```

We can check the solution:

```
m.sol - v {0, 0, 0}
```

We can also use **Solve** if we form an equation with variables:

```
Solve[m.{x, y, z} == v] {{x -> 47/49, y -> 5/49, z -> -3/7}}
```

If a solution does not exist, **LinearSolve** gives a message:

```
LinearSolve[{{1, 2}, {2, 4}}, {2, 1}]
```

`LinearSolve::nosol` : Linear equation encountered which has no solution. More...

`LinearSolve[{ {1, 2}, {2, 4}}, {2, 1}]`

If an infinite number of solutions exist, we are given one of them (with no warning):

`LinearSolve[{ {1, 2}, {2, 4}}, {2, 4}] {2, 0}`

The system may contain symbols:

`LinearSolve[{ {2, 5}, {3, d}}, {3, 2}]`

$$\left\{ \frac{-10 + 3d}{-15 + 2d}, -\frac{5}{-15 + 2d} \right\}$$

3. Conversion between the two forms of linear equations. Consider again the familiar system:

`eqns = {2 x + 5 y + z == 2, 3 x + 4 y + 3 z == 2,
6 x + y + 2 z == 5};`

The corresponding coefficient matrix and constant vector are

`<< LinearAlgebra`MatrixManipulation``
`{m, v} = LinearEquationsToMatrices[eqns, {x, y, z}]`
`{{{2, 5, 1}, {3, 4, 3}, {6, 1, 2}}, {2, 2, 5}}`

If we know **m** and **v**, we can obtain explicit equations as follows (here we use indexed variables):

`eqns = Thread[m.Table[xi, {i, 3}] == v]`
`{2 x1 + 5 x2 + x3 == 2, 3 x1 + 4 x2 + 3 x3 == 2,
6 x1 + x2 + 2 x3 == 5}`

4. Solving several systems with the same coefficient matrix. Suppose that we have one coefficient matrix **m**, but we would like to solve three systems by using the constant vectors **v1**, **v2**, and **v3**:

`m = {{2, 5, 1}, {3, 4, 3}, {6, 1, 2}};`
`v1 = {2, 2, 5}; v2 = {1, 4, 3}; v3 = {4, 2, 6};`

First, we create a function with **LinearSolve**:

`f = LinearSolve[m] LinearSolveFunction[{3, 3}, "<>"]`

Then, we give the three constant vectors as arguments:

`{f[v1], f[v2], f[v3]}`

$$\left\{ \left\{ \frac{47}{49}, \frac{5}{49}, -\frac{3}{7} \right\}, \left\{ \frac{2}{49}, -\frac{5}{49}, \frac{10}{7} \right\}, \left\{ \frac{68}{49}, \frac{26}{49}, -\frac{10}{7} \right\} \right\}$$

In this way we solve three systems, but the PLU decomposition is done only once, thus saving computing time. We could also use **LUdecomposition**:

`lu = LUdecomposition[m];`
`Map[LUBackSubstitution[lu, #] &, {v1, v2, v3}]`

$$\left\{ \left\{ \frac{47}{49}, \frac{5}{49}, -\frac{3}{7} \right\}, \left\{ \frac{2}{49}, -\frac{5}{49}, \frac{10}{7} \right\}, \left\{ \frac{68}{49}, \frac{26}{49}, -\frac{10}{7} \right\} \right\}$$

5. Using the inverse and pseudo-inverse. The inverse of the coefficient matrix can, in principle, be used to solve a linear system:

`m = {{2, 5, 1}, {3, 4, 3}, {6, 1, 2}}; v = {2, 2, 5};`
`Inverse[m].v` $\left\{ \frac{47}{49}, \frac{5}{49}, -\frac{3}{7} \right\}$

However, this method should not be used, since inverting a matrix is a more demanding task than solving a linear system with Gaussian elimination. If the coefficient matrix is singular or nonsquare, we can, however, use a pseudo-inverse to calculate a least-squares solution. (See Sections 5.7, 5.8, and 5.9 for definitions and more information on the pseudo-inverse and least squares solution.) If the system is $Mx = v$, the least-squares solution minimizes the norm $\|Mx - v\|_2$. As an example, consider a system with coefficient matrix **p** and constant vector **c**:

```
p = {{7, 4, 8, 0}, {7, 4, 8, 0}, {1, 8, 3, 3}}; c = {4, 1, 2};
```

The equations do not have any solutions in the usual sense, but a least-squares solution can be calculated:

```
PseudoInverse[p].c {  $\frac{671}{6738}$ ,  $\frac{565}{3369}$ ,  $\frac{1907}{13476}$ ,  $\frac{201}{4492}$  }
```

6. Special methods. **RowReduce** can be used to solve a system of equations. First we form the augmented matrix by appending the constant vector into the rows of the coefficient matrix:

```
m = {{2, 5, 1}, {3, 4, 3}, {6, 1, 2}}; v = {2, 2, 5};  
a = Transpose[Append[Transpose[m], v]]  
{{2, 5, 1, 2}, {3, 4, 3, 2}, {6, 1, 2, 5}}
```

Then, we do Gauss–Jordan elimination to produce the reduced row echelon form; the solution is the last column:

```
g = RowReduce[a]  
{{ {1, 0, 0,  $\frac{47}{49}$ }, {0, 1, 0,  $\frac{5}{49}$ }, {0, 0, 1,  $-\frac{3}{7}$ }}}  
sol = g[All, 4]] {  $\frac{47}{49}$ ,  $\frac{5}{49}$ ,  $-\frac{3}{7}$  }
```

7. Eliminating variables. The following equations contain the parameter **d**, which we would like to eliminate:

```
eqns = {x + y - z == d, x - 2 y + z == -d,  
-x + y + z == 2 d}; vars = {x, y, z};  
Solve[eqns, vars, {d}] {{ {x →  $\frac{3}{5}z$ , y →  $\frac{6}{5}z$ }}}  
Solve::svars : Equations may not give solutions for all solve  
variables. More...
```

Alternatively, we can use **Eliminate**. The result is then expressed as a logical expression:

```
Eliminate[eqns, {d}] 5 x == 3 z && 5 y == 6 z
```

Applications:

1. (Stationary Distribution of a Markov chain) The stationary distribution π (a row vector) of a Markov chain with transition probability matrix P is obtained by using the equations $\pi P = \pi$ and $\sum \pi_i = 1$. One of the equations of the linear system $\pi P = \pi$ can be dropped. We continue Application 1 of Section 73.4:

```
p = {{0.834, 0.166}, {0.398, 0.602}}; st = { $\pi_1$ ,  $\pi_2$ };  
eqns = Append[Most[Thread[st.p == st]], Total[st] == 1]  
{0.834  $\pi_1$  + 0.398  $\pi_2$  ==  $\pi_1$ ,  $\pi_1$  +  $\pi_2$  == 1}  
Solve[eqns] {{ $\pi_1$  → 0.705674,  $\pi_2$  → 0.294326}}
```

So, in the long run, about 71% of days are wet and 29% dry.

73.10 Linear Programming

Commands:

- Solving linear optimization problems written explicitly:
 - Minimize**[{**obj**, **cons**}, **vars**] (Mma 5.0) Give the global minimum of **obj** subject to constraints **cons** with respect to variables **vars**.
 - Maximize**[{**obj**, **cons**}, **vars**] (Mma 5.0) Give the global maximum of **obj** subject to constraints **cons** with respect to variables **vars**.

2. Solving linear optimization problems defined by vectors and matrices:

- **LinearProgramming**[**c**, **a**, **b**] Minimize **c.x** subject to **a.x** $\geq$ **b** and **x** $\geq$ **0**.
- **LinearProgramming**[**c**, **a**, $\{\{\mathbf{b}_1, \mathbf{s}_1\}, \{\mathbf{b}_2, \mathbf{s}_2\}, \dots\}$] The i th constraint is $\mathbf{a}_i.\mathbf{x} \leq \mathbf{b}_i, \mathbf{a}_i.\mathbf{x} == \mathbf{b}_i$, or $\mathbf{a}_i.\mathbf{x} \geq \mathbf{b}_i$ according to whether $\mathbf{s}_i$ is < 0 , $= 0$, or > 0 .

Examples:

1. Solving linear optimization problems written explicitly.

```
obj = 2 x + 3 y;
cons = {x + y ≥ 4, x + 3 y ≥ 5, x ≥ 0, y ≥ 0};
vars = {x, y};
Minimize[{obj, cons}, vars] {17/2, {x → 7/2, y → 1/2}}
```

Here the minimum value of the objective function is 17/2. Next, we solve an integer problem. The constraint **vars** $\in$ **Integers** or **Element**[**vars**, **Integers**] restricts the variables in **vars** to be integer-valued.

```
Minimize[{obj, cons, vars ∈ Integers}, vars]
{9, {x → 3, y → 1}}
```

2. Solving linear optimization problems defined by vectors and matrices. We solve the first problem in Example 1:

```
c = {2, 3}; a = {{1, 1}, {1, 3}}; b = {4, 5};
sol = LinearProgramming[c, a, b] {7/2, 1/2}
c.sol 17/2
```

Applications:

1. (A transportation problem) (See Chapter 50 for definitions and more information about linear programming.) Consider the following transportation problem (cf. [Rus04, pp. 524–526]). Supplies of plants 1 and 2, demands of cities 1, 2, and 3, and costs between the plants and the cities are as follows:

```
supply = {40, 60}; demand = {35, 40, 25};
costs = {{4, 9, 6}, {5, 3, 7}};
```

For example, to transport one unit from plant 1 to city 3 costs \$6. The problem is to minimize the total cost of transportation. Let $x_{i,j}$ be the amount transported from plant i to city j :

```
vars = Table[xi,j, {i, 2}, {j, 3}]
{{x1,1, x1,2, x1,3}, {x2,1, x2,2, x2,3}}
```

The objective function is

```
obj = Flatten[costs].Flatten[vars]
4 x1,1 + 9 x1,2 + 6 x1,3 + 5 x2,1 + 3 x2,2 + 7 x2,3
```

The supply, demand, and nonnegativity constraints are as follows:

```
sup = Thread[Map[Total, vars] ≤ supply]
{x1,1 + x1,2 + x1,3 ≤ 40, x2,1 + x2,2 + x2,3 ≤ 60}
dem = Thread[Total[vars] ≥ demand]
{x1,1 + x2,1 ≥ 35, x1,2 + x2,2 ≥ 40, x1,3 + x2,3 ≥ 25}
non = Thread[Flatten[vars] ≥ 0]
{x1,1 ≥ 0, x1,2 ≥ 0, x1,3 ≥ 0, x2,1 ≥ 0, x2,2 ≥ 0, x2,3 ≥ 0}
```

Then, we solve the problem:

```
sol = Minimize[{obj, sup, dem, non}, Flatten[vars]]
{430, {x1,1 → 15, x1,2 → 0, x1,3 → 25, x2,1 → 20,
      x2,2 → 40, x2,3 → 0}}
```

Appendix

Introduction to Mathematica

1. Executing commands:

- To do a calculation with *Mathematica*, write the command into the notebook document (possibly with the help of a palette) and then execute it by pressing the Enter key in the numeric keypad or by holding down the Shift key and then pressing the Return key.

2. Arithmetic:

- For addition, subtraction, division, and power, use **+**, **-**, **/**, and **^**.
- For multiplication, usually the space key is used, but the ***** key can also be used. For example, **a b** or **a*b** is a product but **ab** is a single variable.

3. Useful techniques:

- To give a name for a result, use **=**; for example, **a = 17^2**.
- To clear the value of a name, use **=.**; for example, **a =.**
- To refer to the last result, use **%**; for example, **% + 18^2**.
- If an input is ended with a semicolon (**a = 17^2;**), the input is processed (after pressing, say, Enter), but the result is not shown in the notebook.
- Input and output labels of the form **In[n]** and **Out[n]** can be turned off with the menu command Kernel ▸ Show In/Out Names.

4. Important conventions:

- All built-in names start with a capital letter.
- The argument of functions and commands are given within square brackets **[]**.
- Parentheses **()** are only used for grouping terms in expressions.

5. Constants and mathematical functions:

- **Pi** (3.141...), **E** (2.718...), **I** ($\sqrt{-1}$), **Infinity**.
- **Sqrt[z]** or **z^(1/2)**, **Exp[z]** or **E^z**, **Log[z]** (the natural logarithm), **Log[b, z]** (a logarithm to base **b**), **Abs[z]**, **Sin[z]**, **ArcSin[z]**, **n!**, **Binomial[n, m]**.

6. Lists:

- A list is formed with curly braces **{ }**; an example: **v = {4, 7, 5}**. A list is simply an ordered collection of elements. Lists can be nested, for example **m = {{4, 7, 5}, {2, 6, 3}}**.
- Parts of lists (and of more general expressions) can be picked up with double square brackets **[[]]**. For example, **m[[2]]** gives **{2, 6, 3}** and **m[[2, 1]]** gives **2**.

7. Useful commands:

- A decimal value can be obtained using **N[expr]**.
- A decimal value to **n** digit precision can be obtained using **N[expr, n]**.
- Numbers near to zero can be set to 0 with **Chop[expr]**.
- An expression can be simplified with **Simplify[expr]**.
- An expression can be expanded out with **Expand[expr]**.
- All commands with a single argument can also be applied with **//**; e.g., **expr // N** means **N[expr]**.

8. Replacements:

- The syntax **expr /. x → a** is used to replace **x** in **expr** with **a**. Here, **x → a** is a transformation rule and **/.** is the replacement operator. The arrow **→** can be written by typing the two characters **->** with no space in between; *Mathematica* transforms **->** to a genuine arrow.

9. Mapping and threading:

- With **Map** we can map the elements of a list with a function, that is, with **Map** we can calculate the value of a function at points given in a list. For example, **Map[f[*#*] &, {a, b, c}]** gives **{f[a], f[b], f[c]}** (a function like **f[*#*] &** is called a pure function). If **f** is a built-in function with one argument, the name of the function suffices. For example, if we want to calculate the 2-norms of the rows of a matrix **m**, we can write **Map[Norm[*#*] &, m]**, but also simply **Map[Norm, m]**.
- With **Thread** we can apply an operation to corresponding parts of two lists. For example, **Thread[{x + y, x - y} == {2, 5}]** gives **{x + y == 2, x - y == 5}**.

10. Functions and programs:

- The syntax **f[x_] := expr** is used to define functions. For example, **f[x_, y_] := x + Sin[y]**.
- In programs, we often use **Module** to define local variables or **With** to define local constants. The syntax is **f[x_] := Module[{local variables}, body]** (similarly for **With**).

11. Loading packages:

- A package is loaded with **<<**. For example, **<<LinearAlgebra`MatrixManipulation`**.

12. Using saved notebooks:

- When you open a saved notebook, *Mathematica* will not automatically process the inputs anew. So, if the notebook contains the definition **a = 17^2** and you also want to use this definition in the new *Mathematica* session, you have to process the input anew; simply put the cursor anywhere in the input and press Enter or Shift-Return.

Getting Help

1. Books, documents, and internet addresses:

- [Wol03]: A complete description of *Mathematica* 5; for linear algebra, see Sections 1.5.7 (Solving Equations), 1.8.3 (Vectors and Matrices), and 3.7 (Linear Algebra).
- [WR99]: A description of the packages of *Mathematica* 4.
- [WR03]: Linear algebra with *Mathematica* 5; a long Help Browser document.
- [Sza00]: Linear algebra with *Mathematica* 3.
- [Rus04]: A general introduction to *Mathematica* 5.
- <http://library.wolfram.com/infocenter/BySubject/Mathematics/Algebra/LinearAlgebra/>: about 100 documents relating to linear algebra with *Mathematica*.

2. Using the Help Browser:

- To open the Help Browser, select the menu command Help ▸ Help Browser.
- In the Help Browser, write a command into the input field and press the Go-button, or choose from the lists of topics.

- In a notebook, write a command like **Eigenvalues** and then press the F1 key (Windows) or Help key (Macintosh) to read the corresponding Help Browser material.
- In a notebook, execute a command like **?Eigenvalues** to get a short description of a command and a link to the more complete information in the Help Browser.

3. Useful material in the Help Browser:

- The *Mathematica* Book ▸ A Practical Introduction to *Mathematica* ▸ Symbolic Mathematics ▸ Solving Equations: Section 1.5.7 of [Wol03].
- The *Mathematica* Book ▸ A Practical Introduction to *Mathematica* ▸ Lists ▸ Vectors and Matrices: Section 1.8.3 of [Wol03].
- The *Mathematica* Book ▸ Advanced Mathematics in *Mathematica* ▸ Linear Algebra: Section 3.7 of [Wol03].
- Built-in Functions ▸ Advanced Documentation ▸ Linear Algebra: [WR03].
- Built-in Functions ▸ Lists and Matrices ▸ Vector Operations: A list of commands related to vectors.
- Built-in Functions ▸ Lists and Matrices ▸ Matrix Operations: A list of commands related to matrices.
- Add-ons and Links ▸ Standard Packages ▸ Linear Algebra: Descriptions of linear algebra packages.

For Users of Earlier Versions

Some commands presented in this chapter are new in *Mathematica* 5.0 or 5.1. Below we have collected information about how to do similar calculations with earlier versions (mainly with 4.2).

- **ArrayPlot[m]**: replace with **ListDensityPlot[Reverse[Abs[m]], ColorFunction → (GrayLevel[1 - #] &), AspectRatio → Automatic, Mesh → False, FrameTicks → False]**.
- **CholeskyDecomposition**: replace with a command of the same name from the **LinearAlgebra`Cholesky`** package.
- **ConjugateTranspose[m]**: replace with **Conjugate[Transpose[m]]**.
- **HessenbergDecomposition**: replace with **Developer`HessenbergDecomposition**.
- **MatrixRank[m]**: replace with **Dimensions[m][[2]] - Length[NullSpace[m]]**.
- **Maximize[{f, cons}, vars]**: replace with **ConstrainedMax[f, cons, vars]**.
- **Minimize[{f, cons}, vars]**: replace with **ConstrainedMin[f, cons, vars]**.
- **Most[v]**: replace with **Drop[v, -1]**.
- **Norm**: Replace with **VectorNorm** or **MatrixNorm** from the **LAMM** package (note: the default is the ∞ -norm).
- **SingularValueDecomposition**: replace with **SingularValues**.
- **SingularValueList[m]**: replace with **SingularValues[m][[2]]**.
- **Total**: Replace with **Apply**. For vectors use **Apply[Plus, v]**. For column sums of matrices, use **Apply[Plus, m]**; for row sums, use **Apply[Plus, Transpose[m]]**; and for the sum of all elements, use **Apply[Plus, Flatten[m]]**.

References

- [BSS93] M.S. Bazaraa, H.D. Sherali, and C.M. Shetty. *Nonlinear Programming: Theory and Algorithms*, 2nd ed. John Wiley & Sons, New York, 1993.

- [Rus04] H. Ruskeepää. *Mathematica Navigator: Mathematics, Statistics, and Graphics*, 2nd ed. Elsevier Academic Press, Burlington, MA, 2004.
- [Sza00] F. Szabo. *Linear Algebra: An Introduction Using Mathematica*. Academic Press, San Diego, 2000.
- [Wol03] S. Wolfram. *The Mathematica Book*, 5th ed. Wolfram Media, Champaign, IL, 2003.
- [WR99] Wolfram Research. *Mathematica 4 Standard Add-on Packages*. Wolfram Media, Champaign, IL, 1999.
- [WR03] Wolfram Research. *Linear Algebra in Mathematica*. In the Help Browser, 2003.

Packages of Subroutines for Linear Algebra

74	BLAS	<i>Jack Dongarra, Victor Eijkhout, and Julien Langou</i>	74-1
	Introduction		
75	LAPACK	<i>Zhaojun Bai, James Demmel, Jack Dongarra, Julien Langou, and Jenny Wang</i>	75-1
	Introduction • Linear System of Equations • Linear Least Squares Problems		
	• The Linear Equality-Constrained Least Squares Problem • A General Linear Model Problem • Symmetric Eigenproblems • Nonsymmetric Eigenproblems		
	• Singular Value Decomposition • Generalized Symmetric Definite Eigenproblems		
	• Generalized Nonsymmetric Eigenproblems • Generalized Singular Value Decomposition		
76	Use of ARPACK and EIGS	<i>D. C. Sorensen</i>	76-1
	The ARPACK Software • Reverse Communication • Directory Structure and Contents • Naming Conventions, Precisions, and Types • Getting Started		
	• Setting Up the Problem • General Use of ARPACK • Using the Computational Modes		
	• MATLAB's EIGS		
77	Summary of Software for Linear Algebra Freely Available on the Web	<i>Jack Dongarra, Victor Eijkhout, and Julien Langou</i>	77-1

74

BLAS

Jack Dongarra

*University of Tennessee and Oakridge
National Laboratory*

Victor Eijkhout

University of Tennessee

Julien Langou

University of Tennessee

74.1 Introduction	74-1
References	74-7

74.1 Introduction

A significant amount of execution time in complicated linear algebraic programs is known to be spent in a few low-level operations. Consequently, reducing the overall execution time of an application program leads to the problem of optimizing these low-level operations. Such low-level optimization are highly machine-dependent and are a matter for specialists. A separation has, therefore, been made by the computational linear algebra community: On the one hand highly efficient, machine-dependent building blocks of linear algebra, the BLAS, (basic linear algebra subprograms) are provided on a given computational platform. On the other hand, linear algebra programs attempt to make the most use of those computational blocks in order to have as good performance as possible across a wide range of platforms.

The first major concerted effort to achieve agreement on the specification of a set of linear algebra kernels resulted in the Level-1 BLAS [LHK79]. The Level-1 BLAS specification and implementation are the result of a collaborative project in 1973 through 1977. The Level-1 BLAS were extensively and successfully exploited by LINPACK [DBM79], a software package for the solution of dense and banded linear equations and linear least squares problems.

With the advent of vector machines, hierarchical memory machines, and shared memory parallel machines, specifications for the Level-2 and -3 BLAS [DDD90a], [DDD90b], [DDH88a], [DDH88b], concerned with matrix–vector and matrix–matrix operations, respectively, were drawn up in 1984 through 1986 and 1987 to 1988. These specifications made it possible to construct new software to utilize the memory hierarchy of modern computers more effectively. In particular, the Level-3 BLAS allowed the construction of software based upon block-partitioned algorithms, typified by the linear algebra software package LAPACK (see Chapter 75).

In this chapter, we present information about the BLAS. We begin with basic definition, then describe the calling sequences, report a number of facts, and finally conclude with a few examples.

Definitions:

Two-dimensional column-major format. A two-dimensional array is said to be in column-major format when its entries are stored by column. This is the way the Fortran language stores two-dimensional

arrays. For example, the declaration `double precision A(2,2)` stores the entries of the 2-by-2 two-dimensional array in the one-dimensional order: `A(1,1), A(2,1), A(1,2), A(2,2)`.

Row-major format. Storage of the entries by row. This is the way C stores two-dimensional arrays.

Packed format. The packed format is relevant for symmetric, Hermitian, or triangular matrices. Half of the matrix (either the upper part or the lower part depending on an UPLO parameter) is stored in a one-dimensional array. For example, if UPLO= 'L', the 3-by-3 symmetric matrix A is stored as `A(1,1), A(2,1), A(3,1), A(2,2), A(3,2), A(3,3)`.

Leading dimension. In column-major format, the leading dimension is the first dimension of a two-dimensional array (as opposed to the dimension of the matrix stored in that array). The leading dimension of an array is necessary to access its elements. For example, if LDA is the leading dimension of A, and A is stored in the two-dimensional column-major format, $A_{i,j}$ is stored in position $i + (j - 1) * LDA$. The leading dimension of an m -by- n matrix is often m . It enables convenient abstraction to access submatrices (see examples). An m -by- n matrix should have $LDA \geq m$.

Increment. The increment of a vector is the number of storage elements from one element to the next. If the increment of a vector is 1, this means that the vector is stored contiguously in memory; an increment of 2 corresponds to using every other array element. The increment is useful to manipulate rows of a matrix stored in column-major format (see Example 3).

BLAS vector description. A vector description in BLAS is defined by three quantities — a vector length, an array or a starting element within an array, and the increment. This gives $(n, X, 1)$ for a vector of size n starting at index X with increment of 1.

BLAS description of a general matrix. A general matrix is described in BLAS by four quantities: a TRANS label ('N', 'T', or 'H'), its dimensions m and n , an array or a starting element within an array, and a leading dimension. This gives $(TRANS, M, N, A, LDA)$ if one wants to operate on the TRANS of an M -by- N matrix starting in `A(1,1)` and stored in an array of leading dimension LDA.

Prefixes The first letter of the name of a BLAS routine indicates the Fortran type on which it operates.

S - REAL	C - COMPLEX
D - DOUBLE PRECISION	Z - COMPLEX*16

Prefixes for Level-2 and Level-3 BLAS

Matrix types:

GE - GEneral	GB - General Band	
SY - SYmmetric	SB - Sym. Band	SP - Sym. Packed
HE - HErmitian	HB - Herm. Band	HP - Herm. Packed
TR - TRIangular	TB - Triang. Band	TP - Triang. Packed

Level-2 and Level-3 BLAS Options

TRANS	= 'No transpose', 'Transpose', 'Conjugate transpose'
UPLO	= 'Upper triangular', 'Lower triangular'
DIAG	= 'Non-unit triangular', 'Unit triangular'
SIDE	= 'Left', 'Right' on the right)

Calls:

- Below is the calling sequence for the GEMV matrix-vector multiply subroutine. The GEMV routine performs the mathematical operation

$$y \leftarrow \alpha Ax + \beta y, \quad \text{or} \quad y \leftarrow \alpha A^T x + \beta y.$$

For double precision, the calling sequence looks like

```
SUBROUTINE DGEMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX,
$                BETA, Y, INCY )
```

The types of the variables are as follows:

```
DOUBLE PRECISION  ALPHA, BETA
INTEGER           INCX, INCY, LDA, M, N
CHARACTER*1       TRANS
DOUBLE PRECISION  A( LDA, * ), X( * ), Y( * )
```

The meaning of the variables is as follows:

TRANS : Specifies the operation to be performed as

TRANS= 'N' $y \leftarrow \alpha Ax + \beta y$,

TRANS= 'T' $y \leftarrow \alpha A^T x + \beta y$.

M : Specifies the number of rows of the matrix A.

N : Specifies the number of columns of the matrix A.

ALPHA: Specifies the scalar α .

A : Points to the first entry of the two-dimensional array that stores the elements of A in column major format.

LDA : Specifies the leading dimension of A.

X : Points to the incremented array that contains the vector x.

INCX : Specifies the increment for the elements of X.

BETA : Specifies the scalar β .

Y : Points to the incremented array that contains the vector y.

INCY : Specifies the increment for the elements of Y.

All variables are left unchanged after completion of the routines, except Y.

2. The following table is the quick reference to the BLAS.

Facts:

1. The BLAS represent fairly simple linear algebra kernels that are easily coded in a few lines. One could also download reference source codes and compile them (<http://www.netlib.org/blas>). However, this strategy is unlikely to give good performance, no matter the level of sophistication of the compiler. The recommended way to obtain an efficient BLAS is to use vendor BLAS libraries, or to install the ATLAS [WPD01] package. Figure 74.1 illustrates the fact that ATLAS BLAS clearly outperform the reference implementation for the matrix–matrix multiply by a factor of five on a modern architecture.
2. To a great extent, the user community has embraced the BLAS, not only for performance reasons, but also because developing software around a core of common routines like the BLAS is good software engineering practice. Highly efficient, machine-specific implementations of the BLAS are available for most modern high-performance computers. To obtain an up-to-date list of available optimized BLAS, see the BLAS FAQ at <http://www.netlib.org/blas>.
3. Level-1 BLAS operates on $\mathcal{O}(n)$ data and performs $\mathcal{O}(n)$ operations. Level-2 BLAS operates on $\mathcal{O}(n^2)$ data and performs $\mathcal{O}(n^2)$ operations. Level-3 BLAS operates on $\mathcal{O}(n^2)$ data and performs $\mathcal{O}(n^3)$ operations. Therefore, the ratio operation/data is $\mathcal{O}(1)$ for Level-1 BLAS and Level-2 BLAS, but $\mathcal{O}(n)$ for Level-3 BLAS. On modern architecture, where memory access is particularly slow compared to computation time, Level-3 BLAS exploit this $\mathcal{O}(n)$ ratio to mask the memory access bottleneck (latency and bandwidth). Figure 74.1 illustrates the fact that Level-3 BLAS on a modern

Level-1 BLAS

	dim	scalar	vector	vector	scalars	A, B, C, S)	5-element array	prefixes
SUBROUTINE xROTG (							Generate plane rotation	S, D
SUBROUTINE xROTMG(					D1, D2, A, B,	PARAM)	Generate modified plane rotation	S, D
SUBROUTINE xROT (N,			X, INCX, Y, INCY,			C, S)	Apply plane rotation	S, D
SUBROUTINE xSWAP (N,			X, INCX, Y, INCY,				Apply modified plane rotation	S, D
SUBROUTINE xSCAL (N,		ALPHA,	X, INCX)				$x \leftrightarrow y$	S, D, C, Z
SUBROUTINE xCOPY (N,			X, INCX, Y, INCY)				$x \leftarrow \alpha x$	S, D, C, Z, CS, ZD
SUBROUTINE xAXPY (N,		ALPHA,	X, INCX, Y, INCY)				$y \leftarrow x$	S, D, C, Z
FUNCTION xDOT (N,			X, INCX, Y, INCY)				$y \leftarrow \alpha x + y$	S, D, C, Z
FUNCTION xDOTU (N,			X, INCX, Y, INCY)				$dot \leftarrow x^T y$	S, D, C, Z
FUNCTION xDOTC (N,			X, INCX, Y, INCY)				$dot \leftarrow x^T y$	S, D, DS
FUNCTION xxDOT (N,			X, INCX, Y, INCY)				$dot \leftarrow x^H y$	C, Z
FUNCTION xNRM2 (N,			X, INCX)				$dot \leftarrow \alpha + x^T y$	SDS
FUNCTION xASUM (N,			X, INCX)				$nrm2 \leftarrow \ x\ _2$	S, D, SC, DZ
FUNCTION IxAMAX (N,			X, INCX)				$asum \leftarrow \ re(x)\ _1 + \ im(x)\ _1$	S, D, SC, DZ
							$amax \leftarrow 1^{st} k re(x_k) + im(x_k) $	S, D, C, Z
							$= \max(re(x_i) + im(x_i))$	

Level-2 BLAS

	dim	b-width	scalar	matrix	vector	scalar	vector	options
xGEMV (	M, N,		ALPHA, A, LDA, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y, y \leftarrow \alpha A^T x + \beta y,$ $y \leftarrow \alpha A^H x + \beta y, A - m \times n$	S, D, C, Z
xGBMV (	M, N, KL, KU,	ALPHA, A, LDA, X, INCX, BETA, Y, INCY)					$y \leftarrow \alpha Ax + \beta y, y \leftarrow \alpha A^T x + \beta y,$ $y \leftarrow \alpha A^H x + \beta y, A - m \times n$	S, D, C, Z
xHEMV (UPLO,	N,		ALPHA, A, LDA, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	C, Z
xHBMV (UPLO,	N, K,		ALPHA, A, LDA, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	C, Z
xHPMV (UPLO,	N,		ALPHA, AP, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	C, Z
xSYMV (UPLO,	N,		ALPHA, A, LDA, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	S, D
xSBMV (UPLO,	N, K,		ALPHA, A, LDA, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	S, D
xSPMV (UPLO,	N,		ALPHA, AP, X, INCX, BETA, Y, INCY)				$y \leftarrow \alpha Ax + \beta y$	S, D
xTRMV (UPLO, TRANS, DIAG,	N,		A, LDA, X, INCX)				$x \leftarrow Ax, x \leftarrow A^T x, x \leftarrow A^H x$	S, D, C, Z
xTBMV (UPLO, TRANS, DIAG,	N, K,		A, LDA, X, INCX)				$x \leftarrow Ax, x \leftarrow A^T x, x \leftarrow A^H x$	S, D, C, Z
xTPMV (UPLO, TRANS, DIAG,	N,		AP, X, INCX)				$x \leftarrow Ax, x \leftarrow A^T x, x \leftarrow A^H x$	S, D, C, Z
xTRSV (UPLO, TRANS, DIAG,	N,		A, LDA, X, INCX)				$x \leftarrow A^{-1} x, x \leftarrow A^{-T} x, x \leftarrow A^{-H} x$	S, D, C, Z
xTBSV (UPLO, TRANS, DIAG,	N, K,		A, LDA, X, INCX)				$x \leftarrow A^{-1} x, x \leftarrow A^{-T} x, x \leftarrow A^{-H} x$	S, D, C, Z
xTPSV (UPLO, TRANS, DIAG,	N,		AP, X, INCX)				$x \leftarrow A^{-1} x, x \leftarrow A^{-T} x, x \leftarrow A^{-H} x$	S, D, C, Z
options	dim	scalar	vector	vector	matrix	vector	matrix	

xGER	(	M, N, ALPHA, X, INCX, Y, INCY, A, LDA)				$A \leftarrow \alpha xy^T + A, A - m \times n$	S, D
xGERU	(	M, N, ALPHA, X, INCX, Y, INCY, A, LDA)				$A \leftarrow \alpha xy^T + A, A - m \times n$	C, Z
xGERC	(	M, N, ALPHA, X, INCX, Y, INCY, A, LDA)				$A \leftarrow \alpha xy^H + A, A - m \times n$	C, Z
xHER	(	N, ALPHA, X, INCX, A, LDA)				$A \leftarrow \alpha xx^H + A$	C, Z
xHPR	(	N, ALPHA, X, INCX, AP)				$A \leftarrow \alpha xx^H + A$	C, Z
xHER2	(	N, ALPHA, X, INCX, Y, INCY, A, LDA)				$A \leftarrow \alpha xy^H + y(\alpha x)^H + A$	C, Z
xHPR2	(	N, ALPHA, X, INCX, Y, INCY, AP)				$A \leftarrow \alpha xy^H + y(\alpha x)^H + A$	C, Z
xSYR	(	N, ALPHA, X, INCX, A, LDA)				$A \leftarrow \alpha xx^T + A$	S, D
xSPR	(	N, ALPHA, X, INCX, AP)				$A \leftarrow \alpha xx^T + A$	S, D
xSYR2	(	N, ALPHA, X, INCX, Y, INCY, A, LDA)				$A \leftarrow \alpha xy^T + \alpha yx^T + A$	S, D
xSPR2	(	N, ALPHA, X, INCX, Y, INCY, AP)				$A \leftarrow \alpha xy^T + \alpha yx^T + A$	S, D
Level-3 BLAS							
	options		dim	scalar matrix	matrix	scalar matrix	
xGEMM	(	TRANSA, TRANSB,	M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)			$C \leftarrow \alpha op(A)op(B) + \beta C,$ $op(X) = X, X^T, X^H, C - m \times n$	S, D, C, Z
xSYMM	(	SIDE, UPLO,	M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)			$C \leftarrow \alpha AB + \beta C, C \leftarrow \alpha BA + \beta C,$ $C - m \times n, A = A^T$	S, D, C, Z
xHEMM	(	SIDE, UPLO,	M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)			$C \leftarrow \alpha AB + \beta C, C \leftarrow \alpha BA + \beta C,$ $C - m \times n, A = A^H$	C, Z
xSYRK	(	UPLO, TRANS,	N, K, ALPHA, A, LDA, BETA, C, LDC)			$C \leftarrow \alpha AA^T + \beta C, C \leftarrow \alpha A^T A + \beta C,$ $C - n \times n$	S, D, C, Z
xHERK	(	UPLO, TRANS,	N, K, ALPHA, A, LDA, BETA, C, LDC)			$C \leftarrow \alpha AA^H + \beta C, C \leftarrow \alpha A^H A + \beta C,$ $C - n \times n$	C, Z
xSYR2K	(	UPLO, TRANS,	N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)			$C \leftarrow \alpha AB^T + \bar{\alpha} B A^T + \beta C,$ $C \leftarrow \alpha A^T B + \bar{\alpha} B^T A + \beta C,$ $C - n \times n$	S, D, C, Z
xHER2K	(	UPLO, TRANS,	N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)			$C \leftarrow \alpha AB^H + \bar{\alpha} B A^H + \beta C,$ $C \leftarrow \alpha A^H B + \bar{\alpha} B^H A + \beta C,$ $C - n \times n$	C, Z
xTRMM	(	SIDE, UPLO, TRANSA,	DIAG, M, N, ALPHA, A, LDA, B, LDB)			$B \leftarrow \alpha op(A)B, B \leftarrow \alpha Bop(A),$ $op(A) = A, A^T, A^H, B - m \times n$	S, D, C, Z
xTRSM	(	SIDE, UPLO, TRANSA,	DIAG, M, N, ALPHA, A, LDA, B, LDB)			$B \leftarrow \alpha op(A^{-1})B, B \leftarrow \alpha Bop(A^{-1}),$ $op(A) = A, A^T, A^H, B - m \times n$	S, D, C, Z

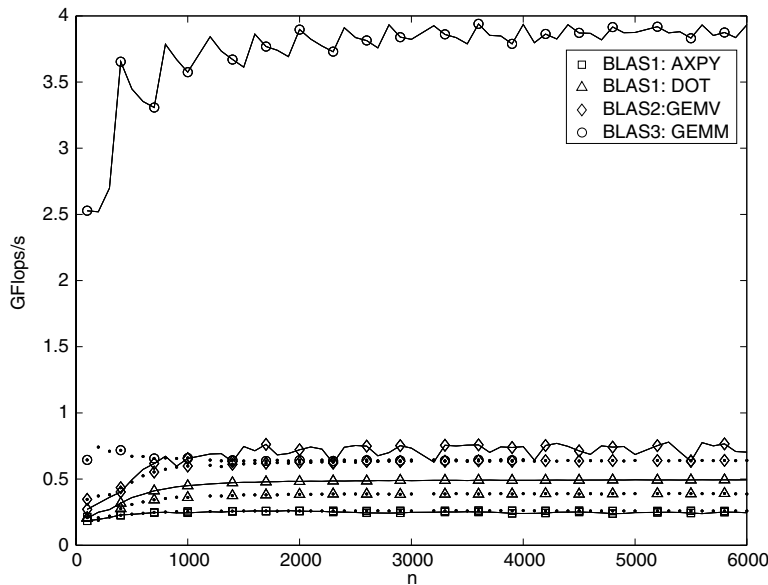


FIGURE 74.1 Performance in GFlops of four BLAS routines for two different BLAS libraries on an Intel Xeon CPU running at 3.20 GHz. The first BLAS library represents an optimized BLAS library (here, ATLAS v3.7.8), its four performance curves are given with a solid line on the graph. The second library represents straightforward implementation (here reference BLAS from netlib); its four performance curves are given with a dotted line on the graph. This graph illustrates two facts: An optimized Level-3 BLAS is roughly five times faster than Level-1 or Level-2 BLAS and it is also roughly five times faster than a reference Level-3 BLAS implementation. To give an idea of the actual time, multiplying two 6000-by-6000 matrices on this machine will take about 2 minutes using the ATLAS BLAS while it will take about 10 minutes using the reference BLAS implementation.

architecture performs 5 times more floating-point operations per second than a Level-2 or Level-1 BLAS routine. Most of the linear algebra libraries try to make as much as possible use of Level-3 BLAS.

4. Most of shared memory computers have a multithreaded BLAS library. By programming a sequential code and linking with the multithreaded BLAS library, the application will use all the computing units of the shared memory system without any explicit parallelism in the user application code.
5. Although here we present only calling sequences from Fortran, it is possible to call the BLAS from C. The major problem for C users is the Fortran interface used by the BLAS. The data layout (the BLAS interface assumes column-major format) and the passage of parameters by reference (as opposed to values) has to be done carefully. See Example 2 for an illustration. Nowadays most BLAS distributions provide a C-interface to the BLAS. This solves these two issues and, thus, we highly recommend its use.
6. The Level-1, Level-2, and Level-3 BLAS are now extended by a new standard with more functionality and better software standard; see [BDD02], [Don02]. For example, the C-interface to the BLAS (see Fact 5) is included in this new BLAS.

Examples:

In all of these examples, A is an m -by- n matrix and it is stored in an M -by- N array starting at position A , B is an n -by- n matrix and it is stored in an N -by- N array starting at position B , C is an m -by- m matrix and it is stored in an M -by- M array starting at position C , X is a vector of size n and it is stored in an N array starting at position X , and Y is a vector of size m and it is stored in an M array starting at position Y . All the two-dimensional arrays are in column-major format, all the one-dimensional arrays have increment one. We assume that m and n are both greater than 11 in Example 3.

1. To perform the operation $y \leftarrow Ax$ the BLAS calling sequence is

```
CALL DGEMV ( 'N', M, N, 1.0D0, A, M, X, 1, 0.0D0, Y, 1 )
```

2. To perform the operation $x \leftarrow \alpha A^T y + \beta x$ the BLAS calling sequence is

```
CALL DGEMV ( 'T', M, N, ALPHA, A, M, Y, 1, BETA, X, 1 )
```

From C, this would give

```
IONE = 1;
dgemv ( "T", &M, &N, &ALPHA, A, &M, Y, &IONE, &BETA, X, &IONE );
```

3. To perform the operation

$$y(2:10) \leftarrow 2A(3:11, 4:11) * B(4, 3:10)^T - 3y(2:10);$$

the BLAS calling sequence is

```
CALL DGEMV ( 'N', 9, 8, 2.0D0, A(3,4), M, B(4,3), N, -3.0D0,
Y(2), 1 )
```

(Note the use of LDA to operate on the submatrix of A and the use of INCX to operate on a row of B.)

4. LAPACK (see Chapter 75) is a library based on the BLAS. Opening its Fortran files enables one to gain a good understanding of how to use BLAS routines. For example, a good way to start is to have a look at `dgetrf.f`, which performs a right-looking LU factorization by making calls to DTRSM and DGEMM.

References

- [BDD02] S. Blackford, J. Demmel, J.J. Dongarra, I. Duff, S. Hammarling, G. Henry, M. Heroux, L. Kaufman, A. Lumsdaine, A. Petitet, R. Pozo, K. Remington, and R.C. Whaley. An updated set of basic linear algebra subprograms (BLAS). *ACM Trans. Math. Softw.*, 28(2):135–151, 2002.
- [Don02] J.J. Dongarra. Basic linear algebra subprograms technical forum standard. *Int. J. High Perform. Appl. Supercomp.*, 16(1–2):1–199, 2002.
- [DBM79] J.J. Dongarra, J.R. Bunch, C.B. Moler, and G.W. Stewart. *LINPACK Users' Guide*. SIAM, Philadelphia, 1979.
- [DDD90a] J.J. Dongarra, J. Du Croz, I.S. Duff, and S. Hammarling. A set of Level 3 basic linear algebra subprograms. *ACM Trans. Math. Softw.*, 16:1–17, 1990.
- [DDD90b] J.J. Dongarra, J. Du Croz, I.S. Duff, and S. Hammarling. Algorithm 679: a set of Level 3 basic linear algebra subprograms. *ACM Trans. Math. Softw.*, 16:18–28, 1990.
- [DDH88a] J.J. Dongarra, J. Du Croz, S. Hammarling, and R.J. Hanson. An extended set of FORTRAN basic linear algebra subprograms. *ACM Trans. Math. Softw.*, 14:1–17, 1988.
- [DDH88b] J.J. Dongarra, J. Du Croz, S. Hammarling, and R.J. Hanson. Algorithm 656: an extended set of FORTRAN basic linear algebra subprograms. *ACM Trans. Math. Softw.*, 14:18–32, 1988.
- [LHK79] C.L. Lawson, R.J. Hanson, D. Kincaid, and F. Krogh. Basic linear algebra subprograms for FORTRAN usage. *ACM Trans. Math. Softw.*, 5:308–323, 1979.
- [WPD01] R.C. Whaley, A. Petitet, and J.J. Dongarra. Automated empirical optimizations of software and the ATLAS project. *Parallel Comp.*, 27:3–35, 2001.

75

LAPACK

Zhaojun Bai

University of California/Davis

James Demmel

University of California/Berkeley

Jack Dongarra

University of Tennessee

Julien Langou

University of Tennessee

Jenny Wang

University of California/Davis

75.1	Introduction	75-1
75.2	Linear System of Equations	75-2
75.3	Linear Least Squares Problems	75-4
75.4	The Linear Equality-Constrained Least Squares Problem	75-6
75.5	A General Linear Model Problem	75-8
75.6	Symmetric Eigenproblems	75-9
75.7	Nonsymmetric Eigenproblems	75-11
75.8	Singular Value Decomposition	75-13
75.9	Generalized Symmetric Definite Eigenproblems	75-15
75.10	Generalized Nonsymmetric Eigenproblems	75-17
75.11	Generalized Singular Value Decomposition	75-20
	References	75-24

75.1 Introduction

LAPACK (linear algebra package) is an open source library of programs for solving the most commonly occurring numerical linear algebra problems [LUG99]. Original codes of LAPACK are written in Fortran 77. Complete documentation as well as source codes are available online at the Netlib repository [LAP]. LAPACK provides driver routines for solving complete problems such as linear equations, linear least squares problems, eigenvalue problems, and singular value problems. Each driver routine calls a sequence of computational routines, each of which performs a distinct computational task. In addition, LAPACK provides comprehensive error bounds for most computed quantities. LAPACK is designed to be portable for sequential and shared memory machines with deep memory hierarchies, in which most performance issues could be reduced to providing optimized versions of the Basic Linear Algebra Subroutines (BLAS). (See Chapter 74).

There have been a number of extensions of LAPACK. LAPACK95 is a Fortran 95 interface to the Fortran 77 LAPACK [LAP95]. CLAPACK and JLAPACK libraries are built using the Fortran to C (f2c) and Fortran to Java (f2j) conversion utilities, respectively [CLA], [JLA]. LAPACK++ is implemented in C++ and includes a subset of the features in LAPACK with emphasis on solving linear systems with nonsymmetric matrices, symmetric positive definite systems, and linear least squares systems [LA+]. ScaLAPACK is a portable implementation of some of the core routines in LAPACK for parallel distributed computing [Sca]. ScaLAPACK is designed for distributed memory machines with very powerful homogeneous sequential processors and with homogeneous network interconnections.

The purpose of this chapter is to acquaint the reader with 10 essential numerical linear algebra problems and LAPACK's way of solving those problems. The reader may find it helpful to consult Chapter 74, where some of the terms used here are defined. The following table summarizes these problems and sections that are treated in version 3.0 of LAPACK.

Type of Problem	Acronyms	Section
Linear system of equations	SV	75.2
Linear least squares problems	LLS	75.3
Linear equality-constrained least squares problem	LSE	75.4
General linear model problem	GLM	75.5
Symmetric eigenproblems	SEP	75.6
Nonsymmetric eigenproblems	NEP	75.7
Singular value decomposition	SVD	75.8
Generalized symmetric definite eigenproblems	GSEP	75.9
Generalized nonsymmetric eigenproblems	GNEP	75.10
Generalized (or quotient) singular value decomposition	GSVD (QSVD)	75.11

Sections have been subdivided into the following headings: (1) Definition: defines the problem, (2) Background: discusses the background of the problem and references to the related sections in this handbook, (3) Driver Routines: describes different types of driver routines available that solve the same problem, (4) Example: specifies the calling sequence for a driver routine that solves the problem followed by numerical examples.

All LAPACK routines are available in four data types, as indicated by the initial letter “x” of each subroutine name: x = “S” means real single precision, x = “D”, real double precision, x = “C”, complex single precision, and x = “Z”, complex*16 or double complex precision. In single precision (and complex single precision), the computations are performed with a unit roundoff of 5.96×10^{-8} . In double precision (and complex double precision) the computations are performed with a unit roundoff of 1.11×10^{-16} .

All matrices are assumed to be stored in column-major format. The software can also handle submatrices of matrices, even though these submatrices may not be stored in consecutive memory locations. For example, to specify the 10-by-10 submatrix lying in rows and columns 11 through 20 of a 30-by-30 matrix A, one must specify

- A(11, 11), the upper left corner of the submatrix
- 30 = Leading dimension of A in its declaration (often denoted LDA in calling sequences)
- 10 = Number of rows of the submatrix (often denoted M, can be at most LDA)
- 10 = Number of columns of submatrix (often denoted N)

All matrix arguments require these 4 parameters (some subroutines may have fewer inputs if, for example, the submatrix is assumed square so that $M = N$). (See Chapter 74, for more details.)

Most of the LAPACK routines require the users to provide them a workspace (WORK) and its dimension (LWORK). The *optimal* workspace dimension refers to the workspace dimension, which enables the code to have the best performance on the targeted machine. The computation of the optimal workspace dimension is often complex so that most of LAPACK routines have the ability to compute it. If a LAPACK routine is called with LWORK=-1, then a workspace query is assumed. The routine only calculates the optimal size of the WORK array and returns this value as the first entry of the WORK array. If a larger workspace is provided, the extra part is not used, so that the code runs at the optimal performance. A *minimal* workspace dimension is provided in the document of routines. If a routine is called with a workspace dimension smaller than the minimal workspace dimension, the computation cannot be performed.

75.2 Linear System of Equations

Definitions:

The problem of linear equations is to compute a solution X of the system of linear equations

$$AX = B, \quad (75.1)$$

where A is an n -by- n matrix and X and B are n -by- m matrices.

Backgrounds:

The theoretical and algorithmic background of the solution of linear equations is discussed extensively in Chapter 37 through Chapter 41, especially Chapter 38.

Driver Routines:

There are two types of driver routines for solving the systems of linear equations — simple driver and expert driver. The expert driver solves the system (Equation 75.1), allows A be replaced by A^T or A^* ; and provides error bounds, condition number estimate, scaling, and can refine the solution. Each of these types of drivers has different implementations that take advantage of the special properties or storage schemes of the matrix A , as listed in the following table.

Data Structure (Matrix Storage Scheme)	Routine Names	
	Simple Driver	Expert Driver
General dense	xGESV	xGESVX
General band	xGBSV	xGBSVX
General tridiagonal	xGTSV	xGTSVX
Symmetric/Hermitian positive definite	xPOSV	xPOSVX
Symmetric/Hermitian positive definite (packed storage)	xPPSV	xPPSVX
Banded symmetric positive definite	xPBSV	xPBSVX
Tridiagonal symmetric positive definite	xPTSV	xPTSVX
Symmetric/Hermitian indefinite	xSYSV/xHESV	xSYSVX/xHESVX
Symmetric/Hermitian indefinite (packed storage)	xSPSV/xHPSV	xSPSVX/xHPSVX
Complex symmetric	CSYSV/ZSYSV	CSYSVX/ZSYSVX

The prefixes GE (for general dense), GB (for general band), etc., have standard meanings for all the BLAS and LAPACK routines.

Examples:

Let us show how to use the simple driver routine SGESV to solve a general linear system of equations. SGESV computes the solution of a real linear Equation 75.1 in single precision by first computing the LU decomposition with row partial pivoting of the coefficient matrix A , followed by the back and forward substitutions. SGESV has the following calling sequence:

```
CALL SGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
```

Input to SGESV:

N: The number of linear equations, i.e., the order of A . $N \geq 0$.

NRHS: The number of right-hand sides, i.e., the number of columns of B . $NRHS \geq 0$.

A, LDA: The N -by- N coefficient matrix A and the leading dimension of the array A .
 $LDA \geq \max(1, N)$.

B, LDB: The N -by- $NRHS$ matrix B and the leading dimension of the array B . $LDB \geq \max(1, N)$.

Output from SGESV:

A: The factors L and U from factorization $A = PLU$; the unit diagonal elements of L are not stored.

IPIV: The pivot indices that define the permutation matrix P ; row i of the matrix was interchanged with row $IPIV(i)$.

B: If $INFO = 0$, the N -by- $NRHS$ solution X .

INFO: $= 0$, successful exit. If $INFO = -j$, the j th argument had an illegal value. If $INFO = j$, $U(j, j)$ is exactly zero. The factorization has been completed, but the factor U is singular, so the solution could not be computed.

Consider a 4-by-4 linear system of Equation (75.1), where

$$A = \begin{bmatrix} 5 & 7 & 6 & 5 \\ 7 & 10 & 8 & 7 \\ 6 & 8 & 10 & 9 \\ 5 & 7 & 9 & 10 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 23 \\ 32 \\ 33 \\ 31 \end{bmatrix}.$$

The exact solution is $\mathbf{x} = [1 \ 1 \ 1 \ 1]^T$. Upon calling SGESV, the program successfully exits with $\text{INFO} = 0$ and the solution X of (75.1) resides in the array B

$$X = \begin{bmatrix} 0.9999998 \\ 1.0000004 \\ 0.9999998 \\ 1.0000001 \end{bmatrix}.$$

Since SGESV performs the computation in single precision arithmetic, it is normal to have an error of the order of 10^{-6} in the solution X . By reading the lower diagonal entries in the array A and filling the diagonal entries with ones, we recover the lower unit triangular matrix L of the LU factorization with row partial pivoting of A as follows:

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.8571429 & 1 & 0 & 0 \\ 0.7142857 & 0.2500000 & 1 & 0 \\ 0.7142857 & 0.2500000 & -0.2000000 & 1 \end{bmatrix}.$$

The upper triangular matrix U is recovered by reading the diagonal and upper diagonal entries in A. That is:

$$U = \begin{bmatrix} 7.0000000 & 10.0000000 & 8.0000000 & 7.0000000 \\ 0 & -0.5714293 & 3.1428566 & 2.9999995 \\ 0 & 0 & 2.5000000 & 4.2500000 \\ 0 & 0 & 0 & 0.1000000 \end{bmatrix}.$$

Finally, the permutation matrix P is the identity matrix that exchanges its i th row with row $\text{IPIV}(i)$, for $i = n, \dots, 1$. Since

$$\text{IPIV} = \begin{bmatrix} 2 & 3 & 4 & 4 \end{bmatrix},$$

we have

$$P = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

75.3 Linear Least Squares Problems

Definitions:

The linear least squares (LLS) problem is to find

$$\min_{\mathbf{x}} \|\mathbf{b} - A\mathbf{x}\|_2, \tag{75.2}$$

where A is an m -by- n matrix and $\mathbf{b}$ is an m element vector.

Backgrounds:

The most usual case is $m \geq n$ and $\text{rank}(A) = n$. In this case, the solution to the LLS problem (75.2) is unique, and the problem is also referred to as finding a least squares solution to an overdetermined system of linear equations. When $m < n$ and $\text{rank}(A) = m$, there are an infinite number of solutions $\mathbf{x}$ that exactly satisfy $\mathbf{b} - A\mathbf{x} = 0$. In this case, it is often useful to find the unique solution $\mathbf{x}$ that minimizes $\|\mathbf{x}\|_2$, and the problem is referred to as finding a *minimum norm solution* to an underdetermined system of linear equations. (See Chapter 5.8 and Chapter 39 for more information on linear least squares problems.)

Driver Routines:

There are four types of driver routines that solve the LLS problem (75.2) and also allow A be replaced by A^* . In the general case when $\text{rank}(A) < \min(m, n)$, we seek the *minimum norm least squares* solution $\mathbf{x}$ that minimizes both $\|\mathbf{x}\|_2$ and $\|\mathbf{b} - A\mathbf{x}\|_2$. The types of driver routines are categorized by the methods used to solve the LLS problem, as shown in the following table.

Type of Matrix	Algorithm	Routine Names
General dense	QR or LQ factorization	xGELS
General dense	Complete orthogonal factorization	xGELSY
General dense	SVD	xGELSS
General dense	Divide-and-conquer SVD	xGELSD

xGELSD is significantly faster than xGELSS, but it demands somewhat more workspace depending on the matrix dimensions. Among these routines, only xGELS requires A to be full rank while xGELSY, xGELSS, and xGELSD allow less than full rank.

Note that all driver routines allow several right-hand side vectors $\mathbf{b}$ and corresponding solutions $\mathbf{x}$ to be handled in a single call, storing these vectors as columns of matrices B and X , respectively. However, the LLS problem (75.2) is solved for each right-hand side independently; that is not the same as finding a matrix X which minimizes $\|B - AX\|_2$.

Examples:

Let us show how to use the simple driver routine SGELS to solve the LLS problem (75.2). SGELS computes the QR decomposition of the matrix A , updates the vector $\mathbf{b}$, and then computes the solution $\mathbf{x}$ by back substitution. SGELS has the following calling sequence:

```
CALL SGELS( TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO )
```

Input to SGELS:

TRANS = 'N' or 'T': solves the LLS with A or A^T .

M, N: The numbers of rows and columns of the matrix A . $M \geq 0$ and $N \geq 0$.

M, NRHS: The number of rows and columns of the matrices B and X . $\text{NRHS} \geq 0$.

A, LDA: The M -by- N matrix A and the leading dimension of the array A , $\text{LDA} \geq \max(1, M)$.

B, LDB: The matrix B and the leading dimension of the array B , $\text{LDB} \geq \max(1, M, N)$.

If TRANS = 'N', then B is M -by- NRHS . If TRANS = 'T', then B is N -by- NRHS .

WORK, LWORK: The workspace array and its dimension. $\text{LWORK} \geq \min(M, N) + \max(1, M, N, \text{NRHS})$.

If LWORK = -1, then a workspace query is assumed; the routine only calculates the optimal size of the WORK array, and returns this value as the first entry of the WORK array.

Output from SGELS:

B: It is overwritten by the solution vectors, stored columnwise.

- If TRANS = 'N' and $M \geq N$, rows 1 to N of B contain the solution vectors of the LLS problem $\min_{\mathbf{x}} \|\mathbf{b} - A\mathbf{x}\|_2$; the residual sum of squares in each column is given by the sum of squares of elements $N + 1$ to M in that column;